
Fundamentals of Software Engineering

Final Project Report

VeriMed

Pharmaceutical Supply Chain & Verification System

Team Members: Umar Asghar (i242532)
Abdul Rehman Nadeem (i242055)
Dawar Ahmed (i242533)

Submitted To: Mr. Atif Saed

Class: DS

Section: B

Submission Date: May 3rd, 2026

Contents

1	Introduction	3
1.1	Project Background	3
1.2	Problem Statement	4
1.3	Objectives	4
1.4	Scope of the Project	5
1.5	Overview of the System	6
2	System Analysis and Requirements	7
2.1	Existing System	7
2.2	Proposed System	7
2.3	Stakeholders Identification	7
2.4	Functional Requirements	8
2.5	Non-Functional Requirements	8
2.6	Feasibility Analysis	9
3	System Design (UML Diagrams)	10
3.1	System Architecture	10
3.1.1	Overall System: Layered (3-Tier) Architecture	10
3.1.2	Backend Communication: Client-Server Architecture	11
3.1.3	Internal Logic: Model-View-Controller (MVC)	11
3.1.4	Data Integration: Data Pipeline	12
3.1.5	Architecture Diagram Components	13
3.2	Use Case Diagram	15
3.3	Use Case Descriptions	15
3.3.1	Use Case 2: Scan Incoming Delivery	16
3.3.2	Use Case 3: Verify Authenticity (Public)	16
3.3.3	Use Case 4: Report Suspicious Location	17
3.4	Activity Diagrams	18
3.5	Sequence Diagrams	24
3.6	Class Diagram	26
3.7	Database Design (ER Diagram / Schema)	27

3.8	Interface Design (Wireframes / Prototypes)	27
4	Implementation	31
4.1	Development Environment	31
4.1.1	Tools and Softwares	31
4.1.2	Programming Languages and Components Development	31
4.2	System Modules Description	31
4.2.1	Manufacturer Module	32
4.2.2	Retailer Module	32
4.2.3	Patient / Verification Module	32
4.3	Frontend Implementation	32
4.4	Backend Implementation	32
4.5	Database Implementation	33
4.6	Integration of System Components	33
4.7	Key Functionalities	33
5	Testing & Evaluation	41
5.1	Testing Strategy	41
5.1.1	Levels of Testing	41
5.1.2	Methodology	41
5.2	Test Cases and Results	42
5.3	System Validation	43
5.4	Performance Evaluation	44
5.5	User Feedback	44
6	Conclusion & Future Work	45
6.1	Summary of the Project	45
6.2	Achievements	45
6.3	Limitations	46
6.4	Future Enhancements	46
A	Source Code Snippets	47
A.1	Backend: Cryptographic Batch & QR Generation (<code>app.py</code>)	47
A.2	Database: FEFO Inventory Sorting Query (<code>database.sql</code>)	48
A.3	Frontend: Real-Time QR Scanner Integration (<code>Retailer.jsx</code>)	49
B	User Manual	50
C	Screens / UI Designs	52
D	Additional Diagrams	58

1 Introduction

1.1 Project Background

The pharmaceutical global supply chain could be described as the most complex of all supply chains. The objective of this supply chain is to ensure that medicines are effectively supplied to the customer. A number of individuals are involved in this process, such as producers, distributors, logistics firms, drug stores, and hospitals. Although this approach enables pharmaceutical companies access to a global market, poor communication remains its major flaw.

Among these issues, the most alarming one is the problem of fake medicines. It is no longer impossible to counter not only expensive medications — nowadays even important medications like antibiotics are prone to being counterfeit products. As per statistics in reports made by the WHO, counterfeit medicines make a significant proportion of the entire pharmaceutical industry. The problem gives rise to treatment failures, health crises, antibiotic resistance, and deaths. It can also cause more damage than any social or human loss. If fake medicine makers copy the packaging of a known company, people will stop trusting that brand. Once trust is lost, it becomes very difficult to regain it.

In the past, tracking such products was done by local computer systems or by writing records by hand. For example, a pharmacy would write down the medicines received from a wholesaler. But there was no way to confirm if the medicine was real or not.

VeriMed is designed to solve this problem. VeriMed is a modern online system that can work on the cloud. It makes the pharmaceutical supply chain clear, secure, and trustworthy. Instead of using separate and disconnected databases, VeriMed uses one digital system. VeriMed keeps a complete record of every step, so the medicine can be tracked safely. The product tracking system is open and transparent. Companies can manage their stock, and people can confirm that the medicine is safe.

1.2 Problem Statement

At the very core of the issue encountered during tracking of pharmaceutical products was the inherent problem with the already implemented design of the system itself, which employs disparate data systems. Right now, the process entails the use of completely independent databases operated by different entities, where important data required for tracking the product is not shared throughout the system. When the medication leaves the original producer, it becomes very difficult to track the product's true source.

There will always be “blind spots” in the supply chain in such a setting. Counterfeiters exploit these very blind spots in order to bring fake medication into the supply chain, at a point where the medication is transferred from secondary wholesalers to local pharmacies. Because most pharmacies are using either paper invoices that can be easily forged, or POS systems which can be manipulated, they cannot verify the medications that are delivered to them.

Moreover, the faulty system renders it impossible for legitimate manufacturers to monitor the entire lifespan of their products; as a result, it becomes unfeasible to recall products selectively. This makes the weakest link in the chain — the consumer — vulnerable, since there is no means by which the consumer can verify whether the drugs they buy are authentic.

1.3 Objectives

The primary objective of the VeriMed project is to build and develop a unique software application that would solve the problem of data silos in the pharmaceutical sector. This main objective is further split into several sub-objectives:

- **Building and designing a role-based web interface:** The web interface would be expected to bring together the three most important entities — pharmaceutical manufacturers, retail pharmacy practitioners, and end users — under one database architecture.
- **Development of a cryptographic QR-code tracking mechanism:** The platform would automatically generate a unique and hash-encrypted QR code for each newly manufactured batch of medicines. These codes will act as the virtual passport for the

physical medicine, documenting its route from production to ultimate distribution.

- **For inventory management of medicines:** The automation system should simplify things for the retail pharmacies, helping them deliver their products through QR codes and manage their inventory according to the FEFO (First-Expired-First-Out) rule, eliminating the possibility of distributing expired medicines and incurring additional expenses.
- **For making information accessible and providing consumers with empowerment:** A consumer-friendly and scanning-enabled portal is required for the VeriMed system. This would help end users confirm the validity, origin, and expiry date of their medicines using their ordinary mobile phones without registering beforehand.

1.4 Scope of the Project

Knowing the exact scope of the VeriMed System is the first step in creating an effective and better software application. In general, the scope of the system involves the full creation of the role-based encryption system that will consist of three related web applications.

The first phase of the project involves the creation of the **Manufacturer Dashboard**. This is where the data life cycle begins. Using the software, drug manufacturers will log into their account, add new batches of drugs, input the chemical composition, set manufacturing and expiry dates, and create QR codes for each batch of medicines.

Second, the scope includes the **Retailer Portal**. This module is designed specifically for the workflow of local pharmacists. It includes the integration of the scanner interface where retailers scan physically delivered shipments and validate the sources of the shipment right away from the central MySQL database, receiving the products electronically. It also includes creating an inventory management dashboard that automatically sorts out locally available products using FEFO principles and informs the pharmacist about products that are about to expire.

Thirdly, the scope includes the creation of the **Patient Interface**. This is a simple and mobile-friendly web interface where no registration from the user's end is required. Any consumer can scan the QR code on the product through their device's camera to obtain validation of authenticity in a visual form, along with the provenance of the product.

Exclusions from Scope: To make this project more feasible and confined to a narrow scope, some features were deliberately excluded, such as physical shipping logistics, GPS tracking, and hardware integration for monitoring storage of medicines. This makes

VeriMed more focused on the software solution.

1.5 Overview of the System

The design of VeriMed uses a common client-server system. The backend of the system is done by Python and the Flask web framework. Flask was chosen as it is simple, has good routing features, and works well with security tools used for QR code generation.

The backend acts as a bridge between the system and the database. The database is organized in 3rd Normal Form (3NF), which means the data is stored in a clean and efficient way.

The frontend is built with HTML5, CSS3, and Bootstrap 5. Pharmacists can use tablets to manage medicines and patients can use their smartphones to scan QR codes. The system is mobile-friendly, working well on phones and easy to use.

The system works by changing the status of a product in steps. First, the manufacturer creates the product and marks it as “Active / Produced.” When different people scan the QR code — such as retailers or customers — the system updates the product’s status in the database. Each update is permanently saved, so it cannot be changed later. This helps prove that the information given to the customer is real.

2 System Analysis and Requirements

2.1 Existing System

Existing systems involve mainly the basic point-of-sale system and the inventory management systems. These systems have the capability to keep track of local stock, but lack the link with the manufacturer's database. The absence of this link makes tracing the drugs and their source very difficult when a shipment arrives, and they rely on the invoice method that could be easily bypassed.

2.2 Proposed System

The proposed system generates a distinct, secure QR code when a batch is manufactured. Once the shipment actually arrives, the retailer scans that exact same barcode into the database to indicate that nothing has changed in the custody of that item. The consumer then scans the very same barcode to open the product's history.

2.3 Stakeholders Identification

- **Manufacturers of Drugs:** They are responsible for producing the drug.
 - **Drug Wholesalers & Retailers:** Those responsible for marketing and distribution of the drug.
 - **Consumers of Drugs:** Those who use the drug for the intended purpose.
 - **IT Managers:** Individuals responsible for management of the VeriMed system.
-

2.4 Functional Requirements

- **FR-01:** The system shall enable authorized Manufacturers to enter details of the formula to create a production batch.
- **FR-02:** The system shall automatically create a unique QR code associated with the Batch ID.
- **FR-03:** The system shall enable Retailers to scan the QR codes and update the inventory automatically through FEFO.
- **FR-04:** The system shall enable users to scan QR codes to check for authenticity without logging in.

2.5 Non-Functional Requirements

Security

- **Data Protection:** All supply chain data must be handled in such a manner that all sensitive information remains confidential and the integrity of such data is maintained. No passwords should ever be stored in a database without being cryptographically hashed first, so that access to the database does not grant access to user accounts.
- **Ensuring Anti-Forgery:** Hash encoding for the QR codes generated for the respective batches of medicine must be secured in a random manner such that they cannot be predicted, copied, or tampered with by any counterfeit manufacturer.
- **Role-Based Access Control:** The system must enforce rigid access boundaries — only a validated Manufacturer can create batches and input quantities, and none but an authenticated Retailer has permission to update inventory states.

Performance and Responsiveness

- **Query Latency:** The query must have very low latency to make real-time feasible. In particular, the `/verify` endpoint for public-facing QR code verification queries must process and respond with authenticity results to the end-user in less than 3 seconds.
- **Database Efficiency:** The read/write operations of the FEFO-trigger service and its associated data leading to the Retailer dashboard must operate with extremely low latency in order to accommodate high-throughput pharmacy environments.

Usability and Accessibility

- **Mobile-First Design:** As retail pharmacists and public consumers will mostly access the system through handheld devices, all frontend interfaces (HTML5/Bootstrap 5) must be responsive and mobile-screen-optimized.
- **Intuitive Workflow:** The verification scanner must require no training and no registration of an account to be used, so that verifying medicines is easy and fast.

Reliability and Availability

- **System Uptime:** The architecture should be capable of supporting many users accessing, scanning, and entering data at the same time.
- **Importance of Error Handling:** Appropriate and understandable error messages must be sent out, for example when a scan fails, so the user knows what happened.

Data Integrity and Maintainability

- **Database Normalization:** The database is written in MySQL in 3NF. This helps to avoid redundancy and update anomalies.
- **Framework Standards:** The backend should be written in such a way that the system becomes flexible for the future, ensuring scaling, maintainability, and a good structure.
- **Role-Based Access Control:** There should be control over the functionalities which can be performed by a certain role only — for example, a user logged in with the role of manufacturer can generate batches.

2.6 Feasibility Analysis

- **Technical Feasibility:** We chose tools like Python, Flask, MySQL, and React, which are easy to integrate, helping make secure codes and good database connections.
- **Economic Feasibility:** Since this was an initial phase, we did not need any paid tools, hardware, or cloud storage. We made use of free tools to achieve this project.
- **Operational Feasibility:** Since the website will be highly responsive, with an easy User Interface and big buttons suited to any age group, no excessive computer skills are needed, thus this website will be easy to use.

3 System Design (UML Diagrams)

3.1 System Architecture

VeriMed has a Layered + Integrated Client-Server model. This architecture was selected in order to make sure that the UI is responsive, with the backend handling the QR code and validation logic for medicines while tracking.

3.1.1 Overall System: Layered (3-Tier) Architecture

The layered architecture helps in separating modules, which makes the system easy to manage and more flexible.

- **Layer 1 — Frontend:** In order to develop the UI, languages such as React and HTML are used. Since they are universal languages, they help make the system responsive for every possible device, such as phones or PCs.
- **Layer 2 — Backend:** For the backend, Python was chosen with Flask due to its easy access to libraries which helped a lot to integrate QR codes. This model is light and fast.
- **Layer 3 — Database:** Developed using MySQL in 3NF form, which helps increase data integrity and reduce anomalies.

Justification:

- **Modular Structure:** Decoupling the frontend from the backend is really helpful. It allows the team to refine the system without interfering with parts of the system that are being used, keeping the backend doing its own thing without any issues.
- **Secure Scalability:** The application layer acts as a bridge to the MySQL database. It helps keep the persistence layer away from the frontend, meaning the frontend does

not get to access the persistence layer directly.

3.1.2 Backend Communication: Client-Server Architecture

From a technical perspective, the project is structured using the classical Client-Server architecture where the communication protocol employed is RESTful API.

- **Client:** The browser of the user makes requests for performing specific functions like creating a new batch or scanning the QR code.
- **Server:** The Flask application written using Python acts as the medium between the client and the MySQL database.

Justification:

- **Efficient Data Exchange:** Using this architecture guarantees the efficiency of data exchange, which directly contributes to the objective of resolving all queries from QR codes within 3 seconds.
- **Secure Operation Isolation:** Such architecture will ensure isolation of operations requiring protection, such as encryption and calculation of FEFO sequence, from other functions.

3.1.3 Internal Logic: Model-View-Controller (MVC)

In order to structure the backend code, an internal routing is done based on the MVC architecture pattern.

- **Model:** Defines the structures for the data which are linked to the tables within the MySQL database (users, batches, inventory).
- **View:** The view consists of the highly responsive HTML pages based on Bootstrap 5 that are visible to the end-user, for example the Manufacturer Dashboard and Retailer Portal.
- **Controller:** The controller consists of the routes from the Flask API that receive user inputs, validates the inputs using logic, works with the Model layer, and informs the View what to do next.

Justification:

- **Workstream Separation:** The separation of the View from the Model makes it easier for the team to enhance the UI without affecting the backend Python code's architecture.
- **Request Centralization:** Only the Controller can accept user requests. This com-

ponent verifies user input data such as manufacturing dates and formula details before passing them to the database.

3.1.4 Data Integration: Data Pipeline

- **Step 1 — Trigger:** The Manufacturer inputs formula details and dates to generate a production batch via the web form.
- **Step 2 — Hash Generation:** The backend makes a unique, safe, and hash-encrypted QR code that is linked to the Batch ID.
- **Step 3 — Storage:** The MySQL database safely stores the generated batch record and its corresponding cryptographic hash.
- **Step 4 — Validation:** After the patient/retail store owner has scanned the QR code, the user sends the hash value to the main server, which confirms whether there is a match by checking the database and if so returns the history of the product.

Justification:

- **System Reliability:** Putting these processes through a strict pipeline makes sure that the information provided to the end-user is correct and prevents any kind of false data from entering the chain.

3.1.5 Architecture Diagram Components

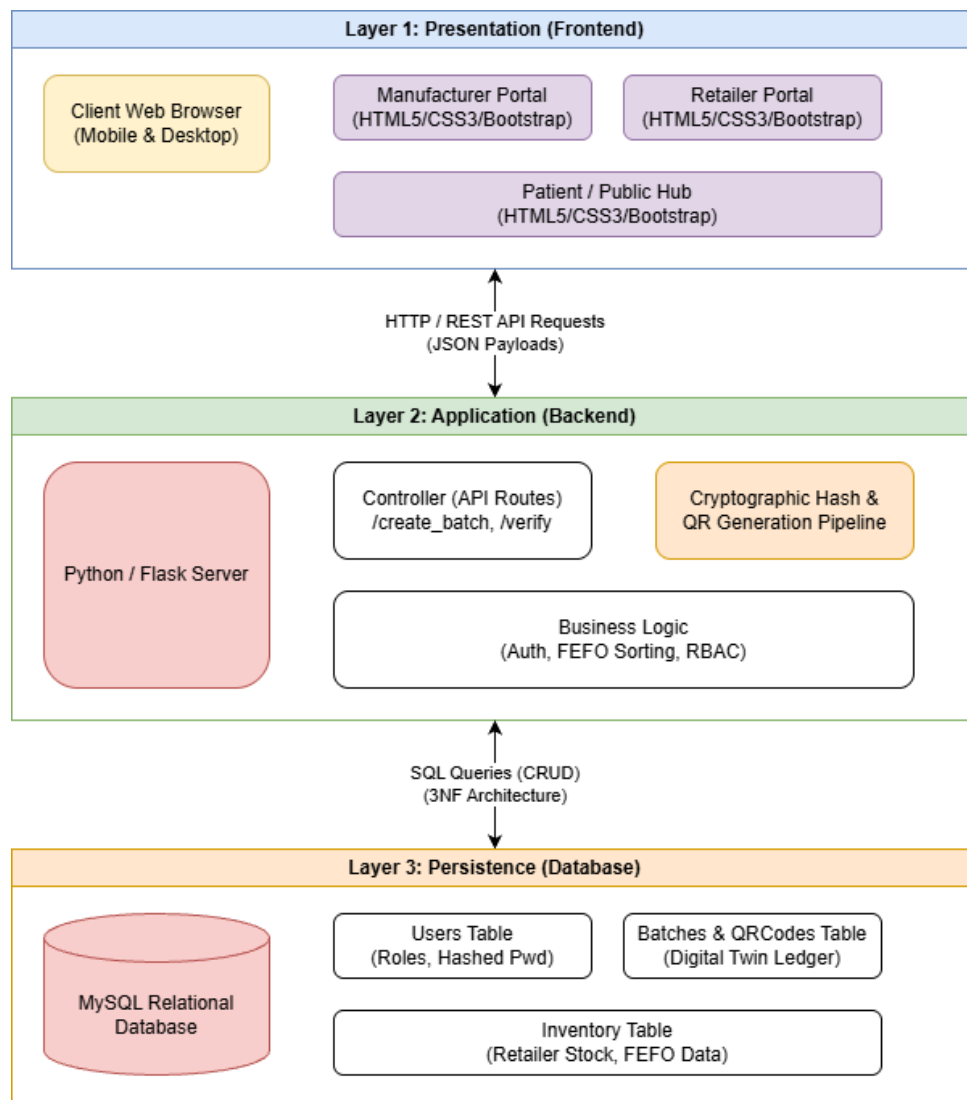


Figure 3.1: Figure 3-A: Layer Diagram

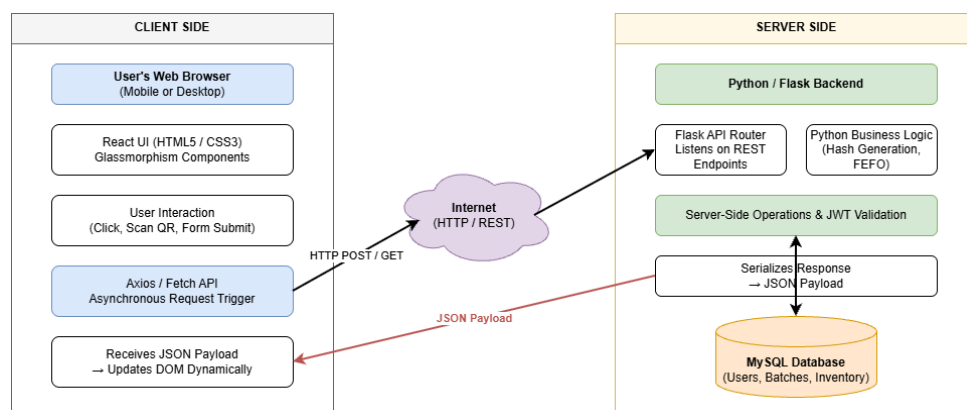


Figure 3.2: Figure 3-A: Layer Diagram (Detail)

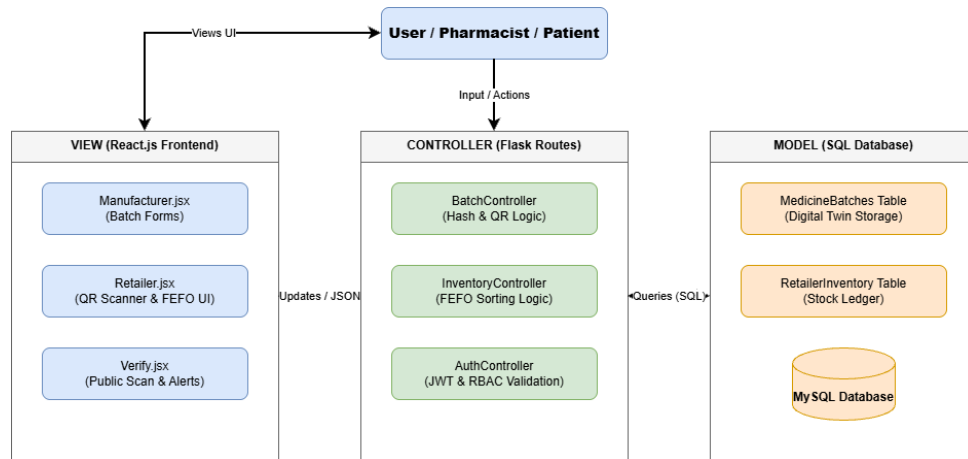


Figure 3.3: Architecture Overview

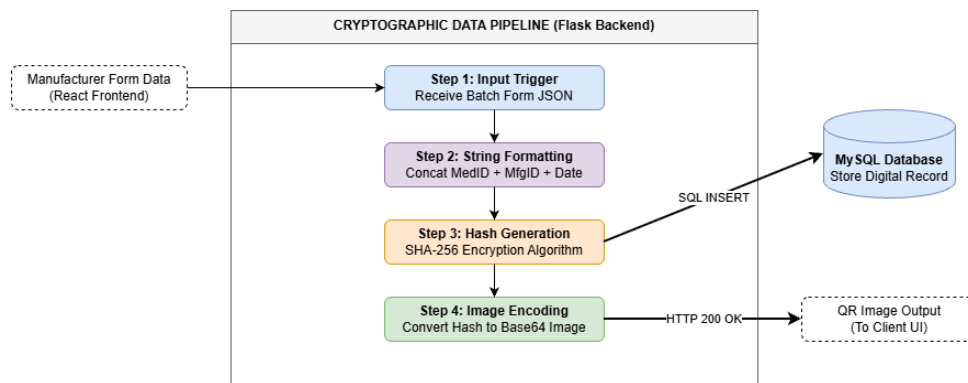


Figure 3.4: Figure 3-B: Architecture Diagram

3.2 Use Case Diagram



Figure 3.5: Use Case Diagram — VeriMed System

3.3 Use Case Descriptions

Use Case 1: Generate New Batch

- **Actor:** Manufacturer
- **Purpose:** Creation of the electronic record for a newly produced batch of medicine and generation of its QR code.
- **Precondition:** Manufacturer is signed into the Manufacturer Portal.
- **Process:**
 1. Manufacturer goes to the Create Batch section.

2. The Manufacturer enters all the necessary details about the newly created batch, for example medicine name and composition.
 3. The Manufacturer clicks “Generate & Create Batch.”
 4. The system validates the entered data.
 5. The system calculates a unique hash value to serve as the identifier of the batch.
 6. The system stores the created batch in the MySQL database.
 7. The system generates a visual QR code that can be downloaded by the user, which contains an encrypted Batch ID.
- **Postconditions:** A new batch has been successfully created and is waiting to be dispatched/scannable by the retailer.

3.3.1 Use Case 2: Scan Incoming Delivery

- **Actor:** Retailer (Pharmacist)
- **Goal:** To digitally receive a delivery of a medicine batch from the manufacturer and then add it to the local pharmacy/retailer’s inventory automatically.
- **Pre-requisites:** Before scanning the delivery, the Retailer should be logged into the Retailer Portal and should have access to a device camera to scan the QR code.
- **Main Scenario:**
 1. The Retailer taps on the “Receive Delivery” button from their dashboard.
 2. Camera access request is prompted; scanner is displayed.
 3. The Retailer uses the camera to scan the QR code on the delivery package of the medicine.
 4. The QR hash is decoded by the system, which further queries the central database.
 5. The system verifies that it is an authorized delivery and fetches batch information (batch quantity, expiry date).
 6. The Retailer confirms quantity received and taps on “Add to Inventory.”
 7. The system updates the database and marks the batch status as “At Retailer.”
 8. The system updates the Retailer’s inventory dashboard automatically, sorting the medicines according to FEFO logic.
- **Alternate Scenario:** If the verification fails (for example, wrong credentials), a warning message will display and then the retailer will not be able to add the batch to the inventory.

3.3.2 Use Case 3: Verify Authenticity (Public)

- **Actor:** Patient / Customer

- **Objective:** To confirm if the medicine they bought is real and safe to ingest without having to create a personal account.
- **Precondition:** The Patient should have a smartphone or computer with an internet connection.
- **Primary Flow of Events:**
 1. The Patient accesses the VeriMed official website and goes to the “Verify Your Medicine” section.
 2. The Patient decides to scan the medicine via camera or type in the ID number found on the packaging.
 3. The Patient either scans the QR Code or types the ID.
 4. The system automatically searches the database for a match between hash/ID.
 5. The system provides a visual outcome (Green for Real, Red for Fake).
 6. If confirmed real, the system shows the complete provenance of the product such as its name, production date, and expiration.
- **Postconditions:** The Patient is provided with the knowledge regarding their medicine’s authenticity and safety.

3.3.3 Use Case 4: Report Suspicious Location

- **Actor:** Patient / Customer
- **Purpose:** To warn the community as well as the concerned authority about fake drugs being sold from a particular pharmacy.
- **Prerequisites:** The Patient is notified by the system that their scanned drug was found to be “Batch Not Found” / counterfeit.
- **Primary Flow:**
 1. After the Patient has undergone an authenticity check for the product, they are informed through a pop-up message containing a “Report This Location” button.
 2. The Patient presses the button in order to fill up the reporting form.
 3. Patient fills out the field regarding the location where they purchased the drug (e.g. “Raja Bazaar, Rawalpindi”) and adds a description (optional).
 4. The patient presses the submit button.
 5. The system adds the location to the database under “Under Review” status.
 6. A message of confirmation is displayed and a link is provided to see the community-reported locations map.
- **Postconditions:** The location is added to the database and marked on the “Fake Drugs Location” board.

3.4 Activity Diagrams

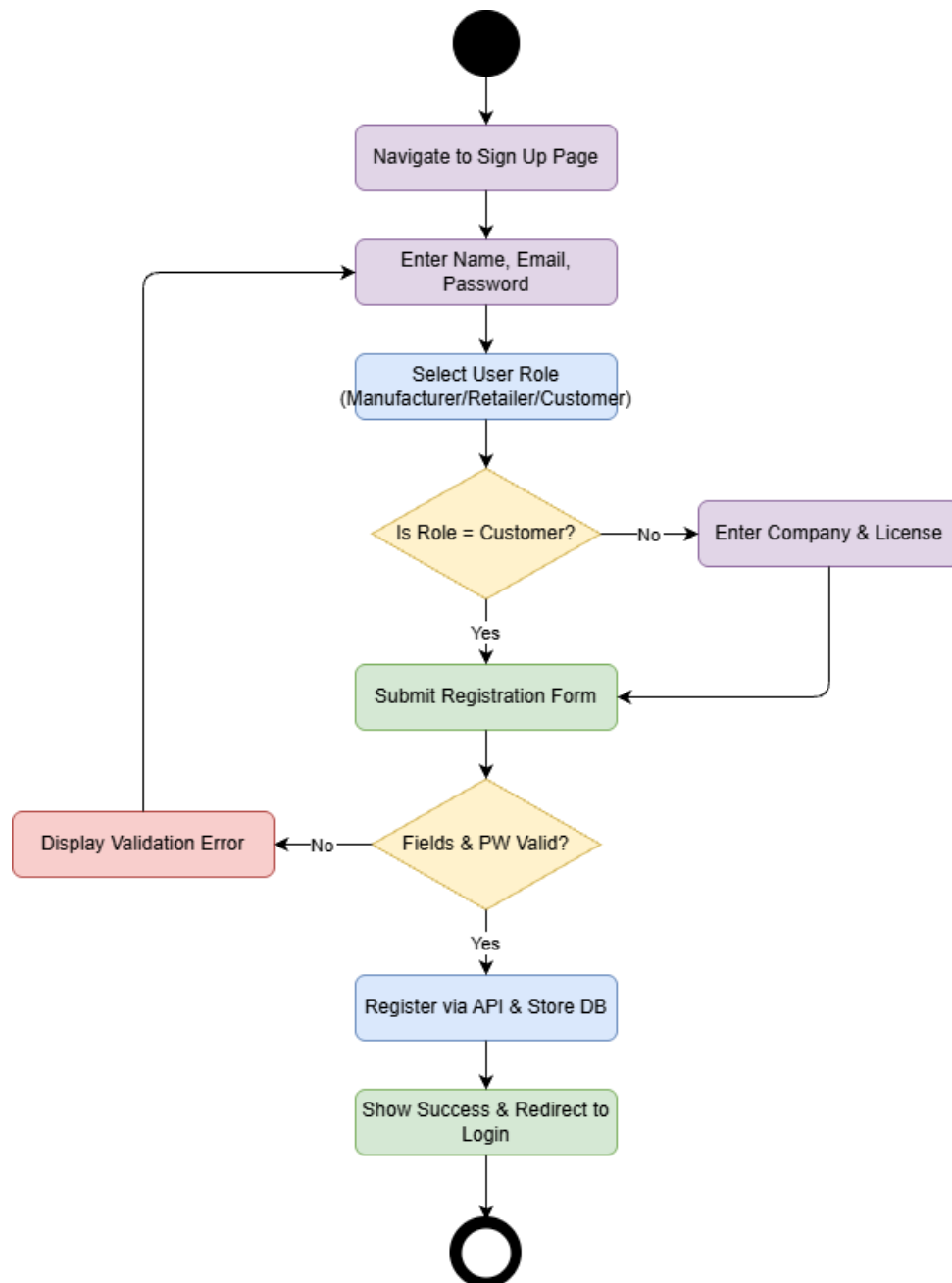


Figure 3.6: Activity Diagram — Batch Generation Flow

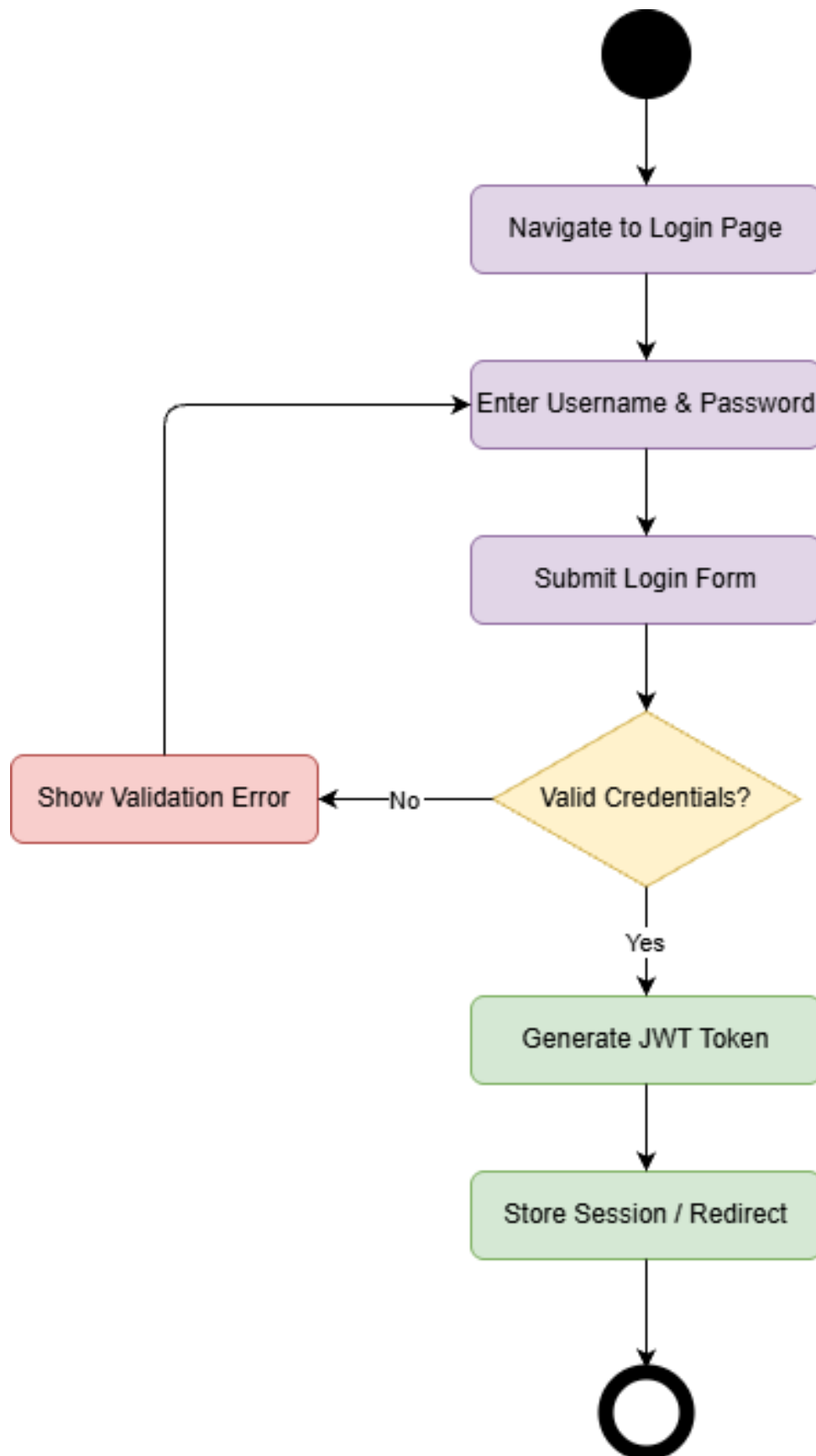


Figure 3.7: Activity Diagram — QR Scan Flow

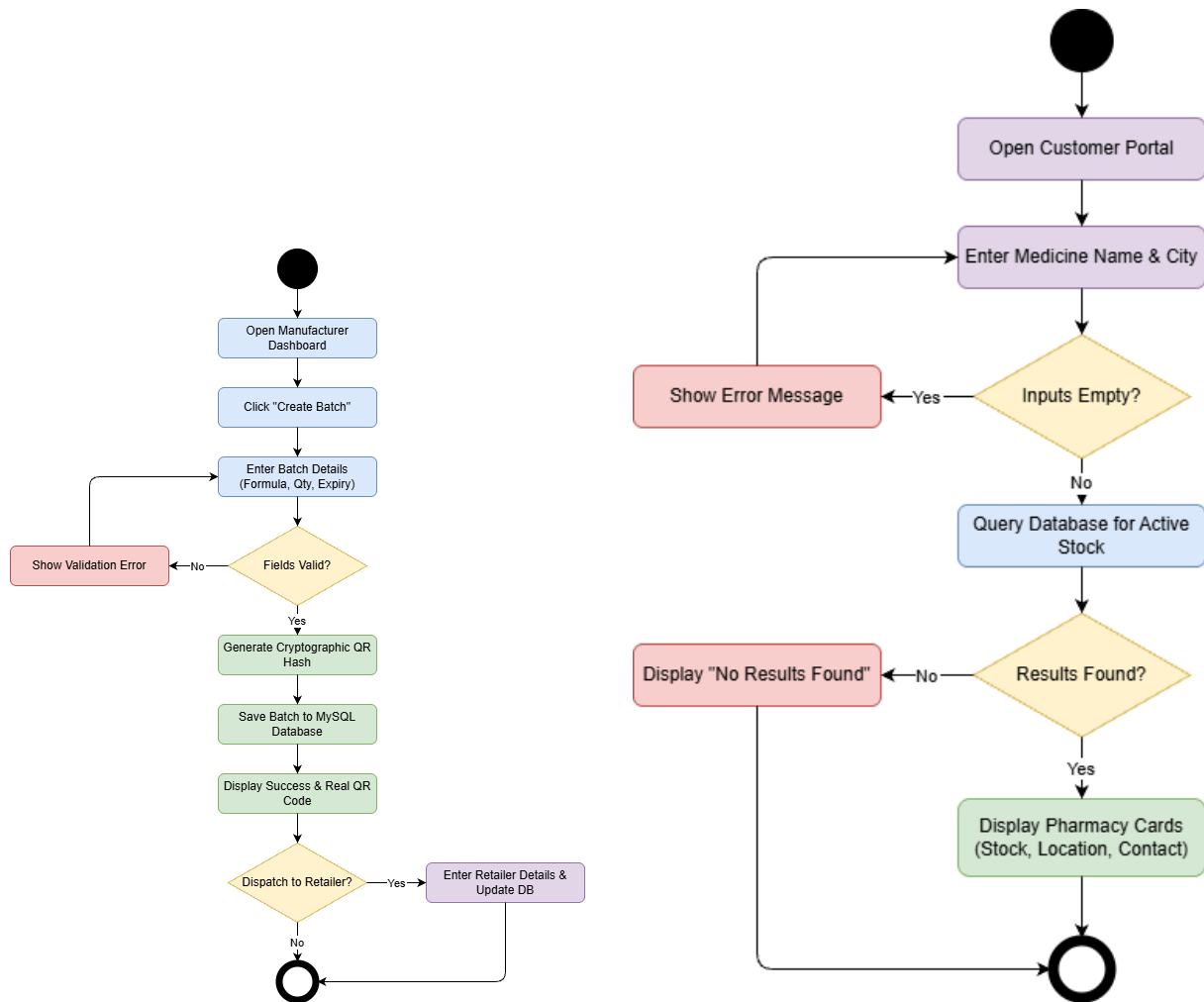


Figure 3.8: Activity Diagrams — Retailer and Patient Flows

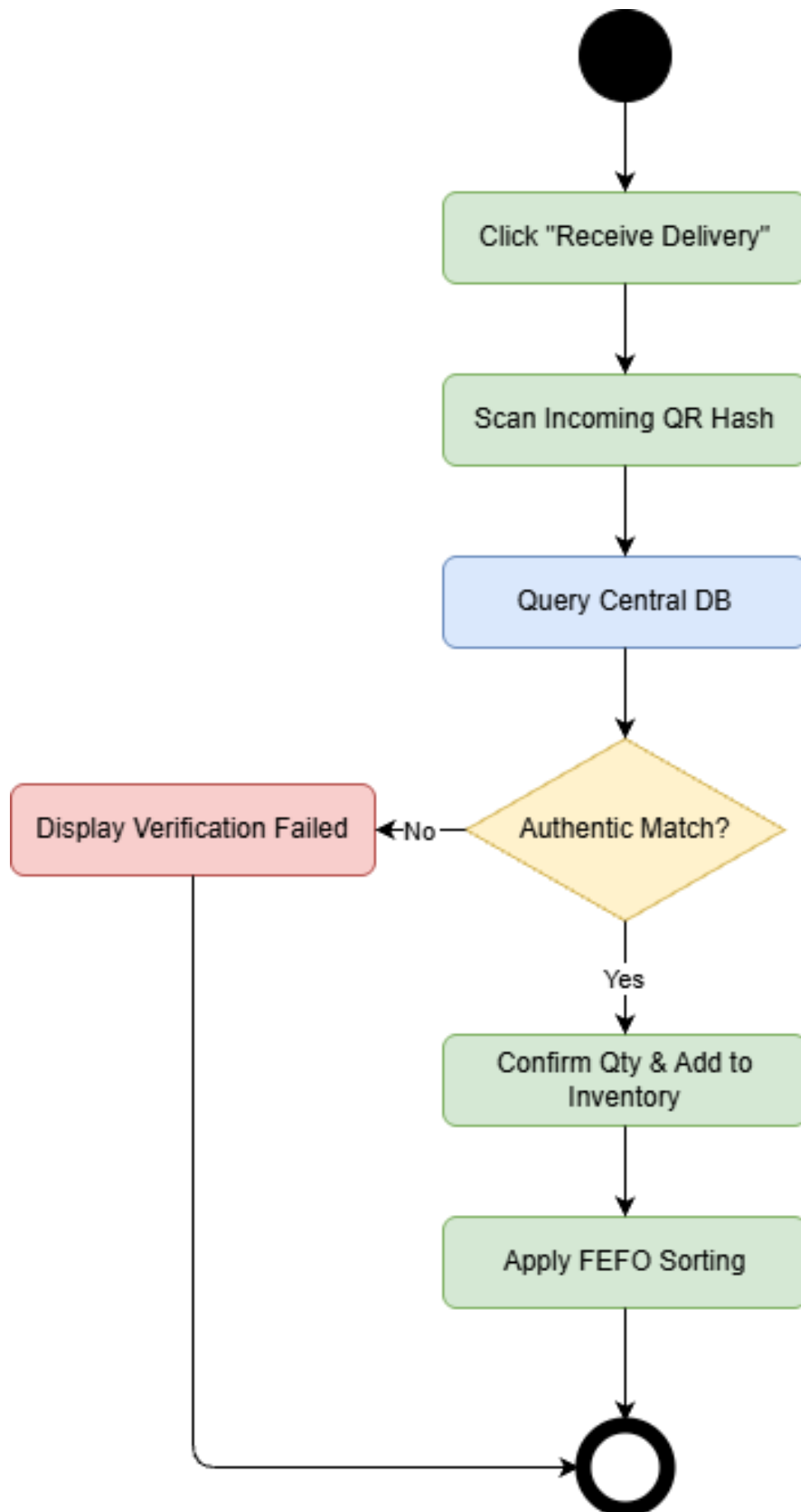


Figure 3.9: Activity Diagram – Verification Flow

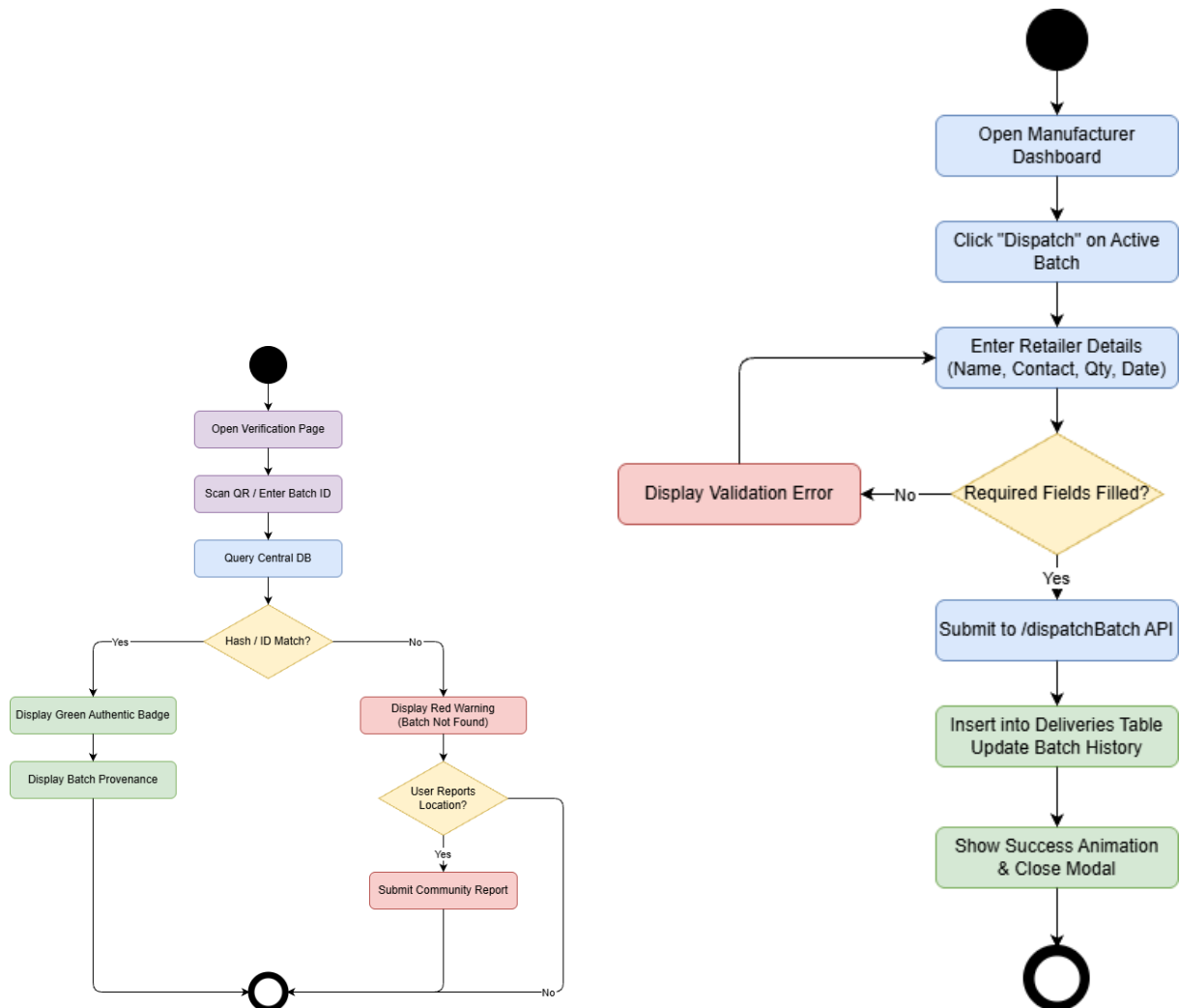


Figure 3.10: Activity Diagrams — Reporting and Authentication Flows

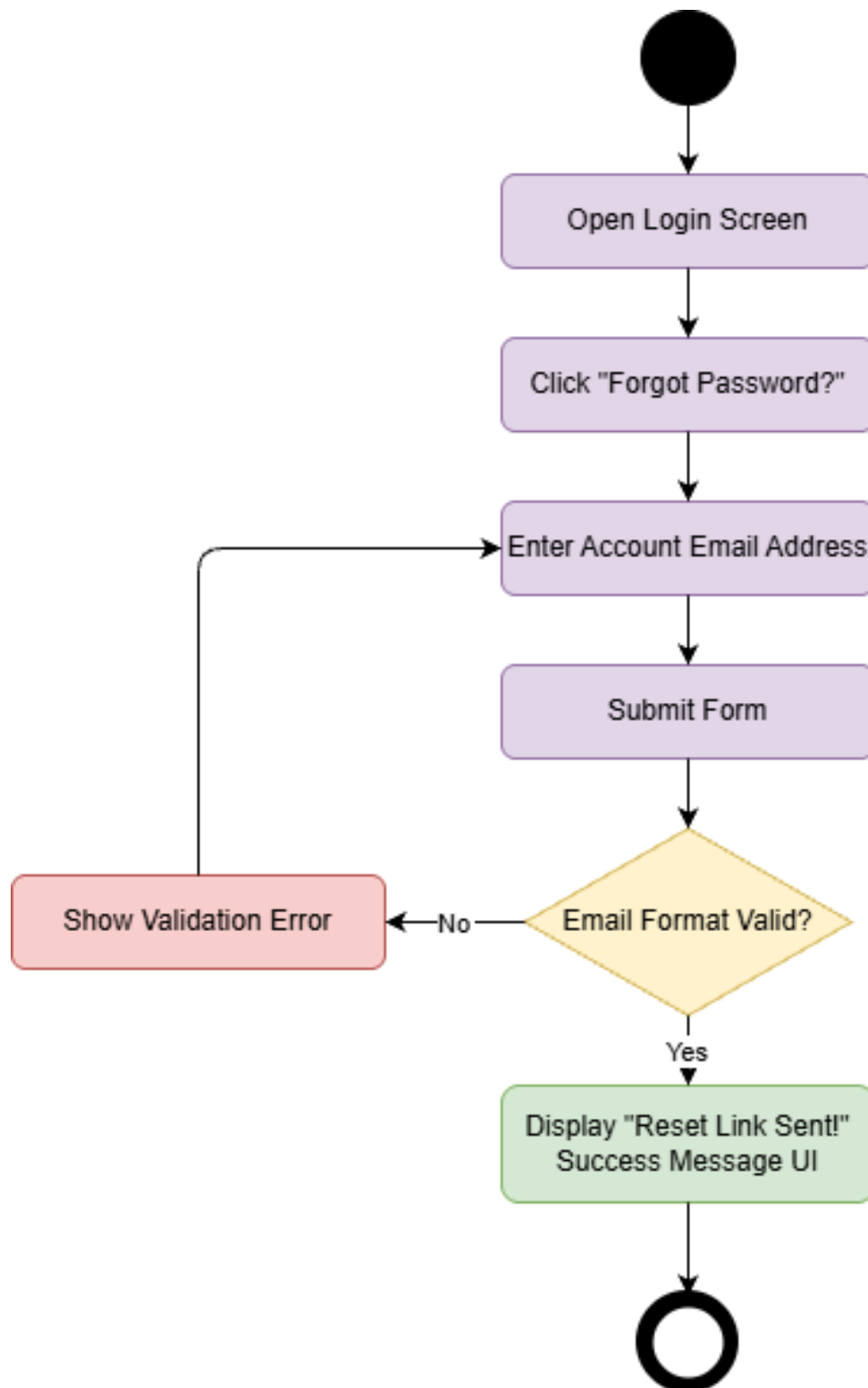


Figure 3.11: Activity Diagram — System Integration Flow

3.5 Sequence Diagrams

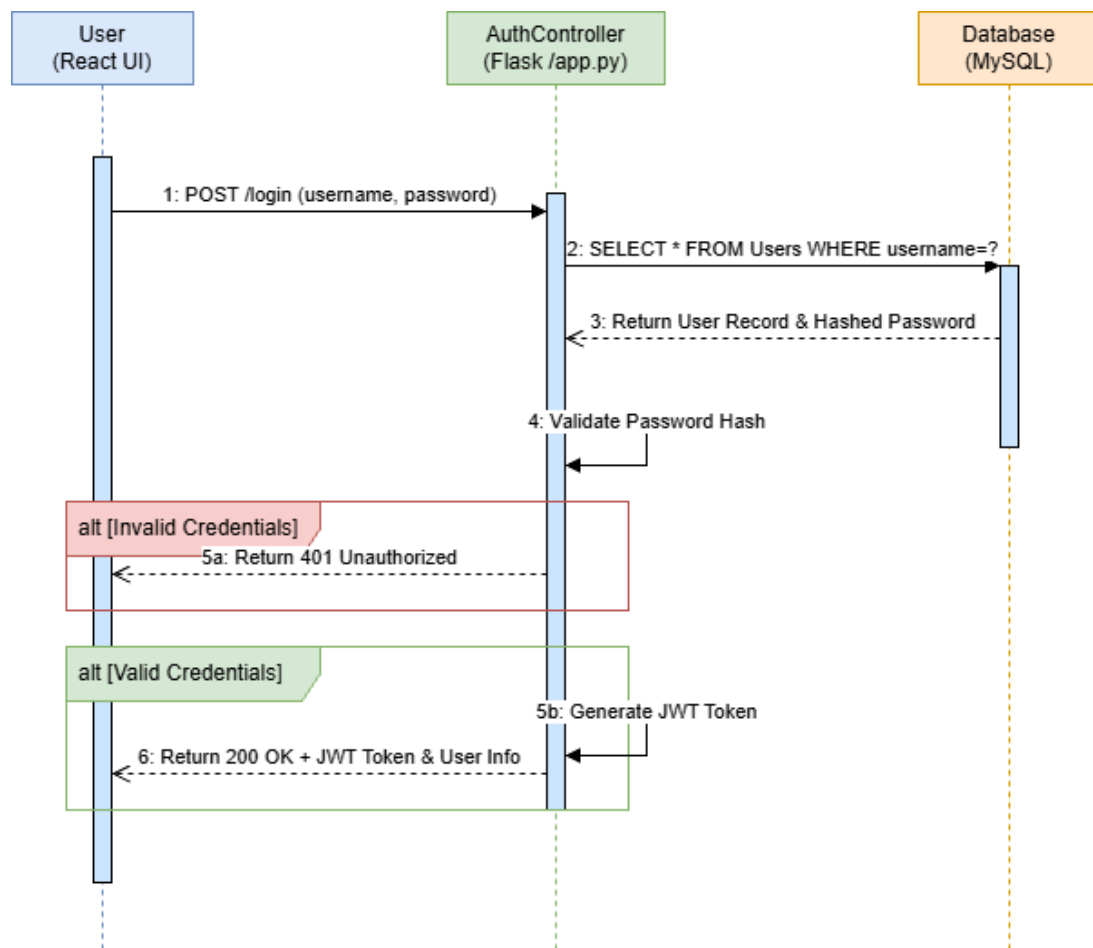


Figure 3.12: Sequence Diagram — Batch Creation

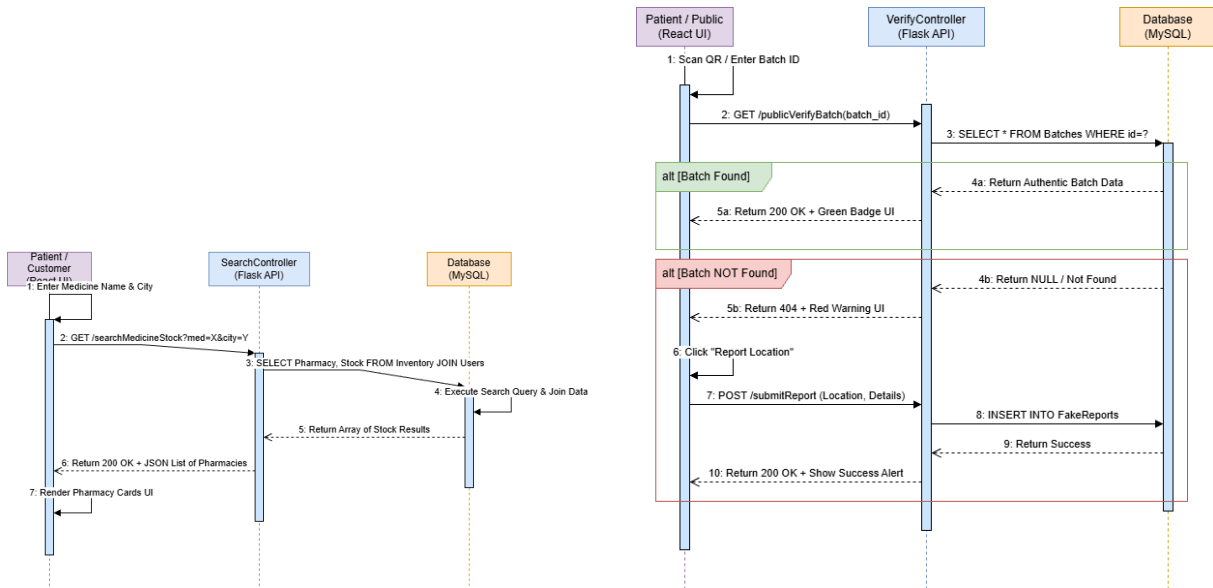


Figure 3.13: Sequence Diagrams — Delivery Scan and Public Verification

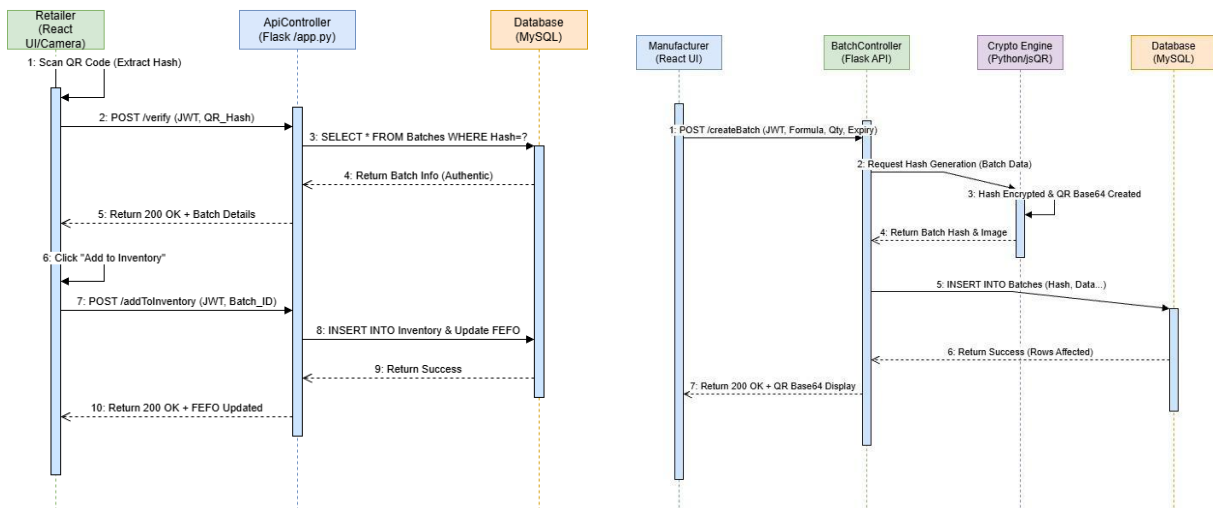


Figure 3.14: Sequence Diagrams — FEFO Inventory and Report Flow

3.6 Class Diagram

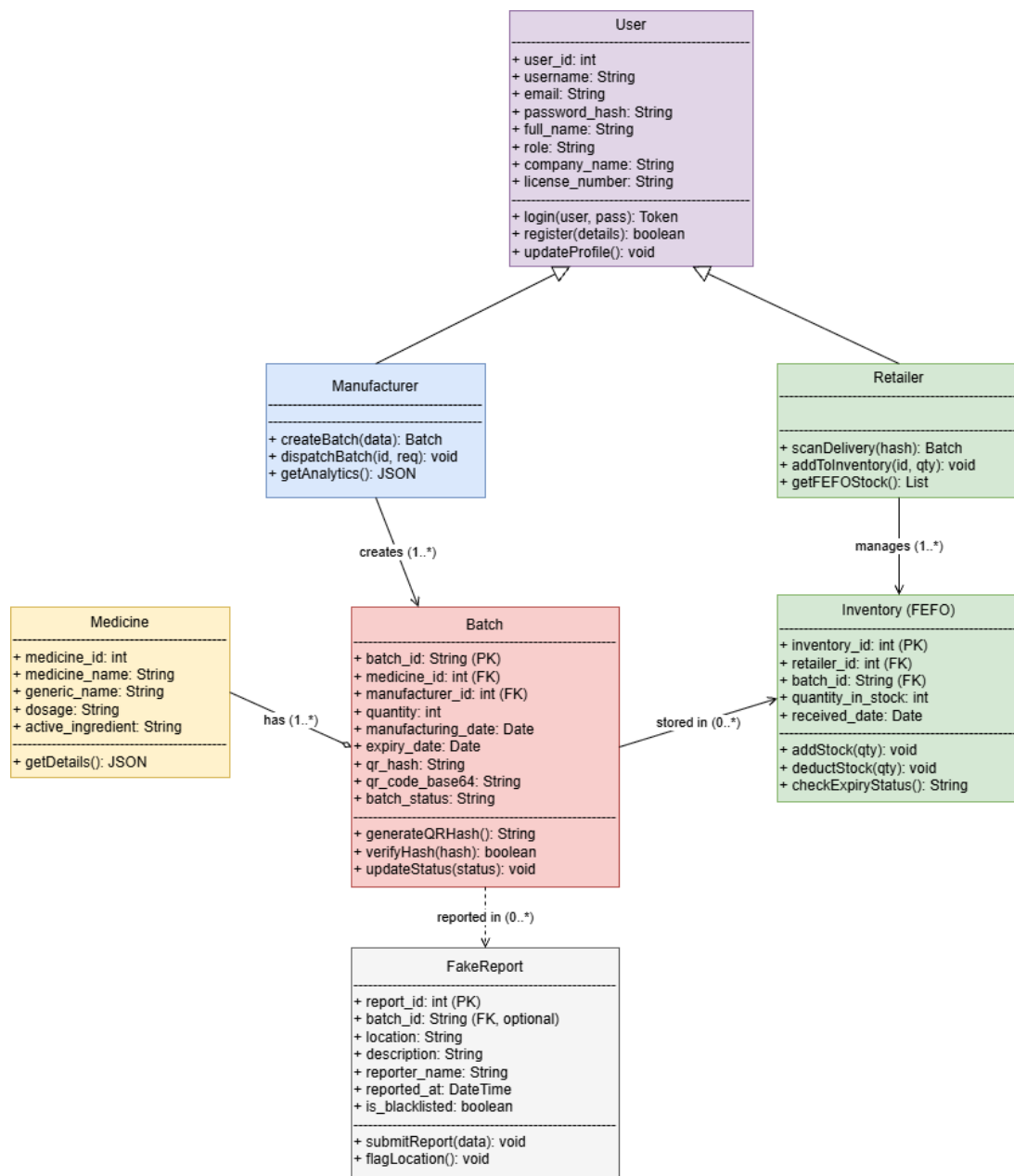


Figure 3.15: Class Diagram — VeriMed System

3.7 Database Design (ER Diagram / Schema)

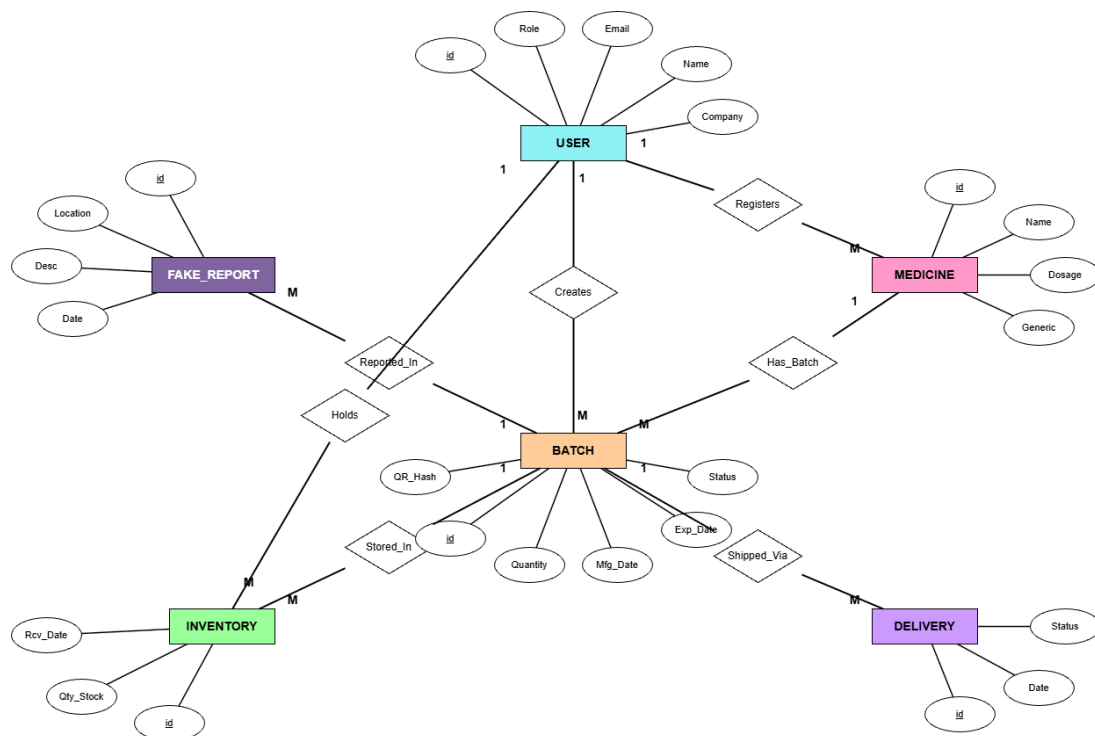


Figure 3.16: Entity-Relationship (ER) Diagram — VeriMed Database

3.8 Interface Design (Wireframes / Prototypes)

Figma Prototype Link: <https://www.figma.com/site/uLuF2HHAdqqk0zU2yATWxM/Untitled?node-id=0-1>

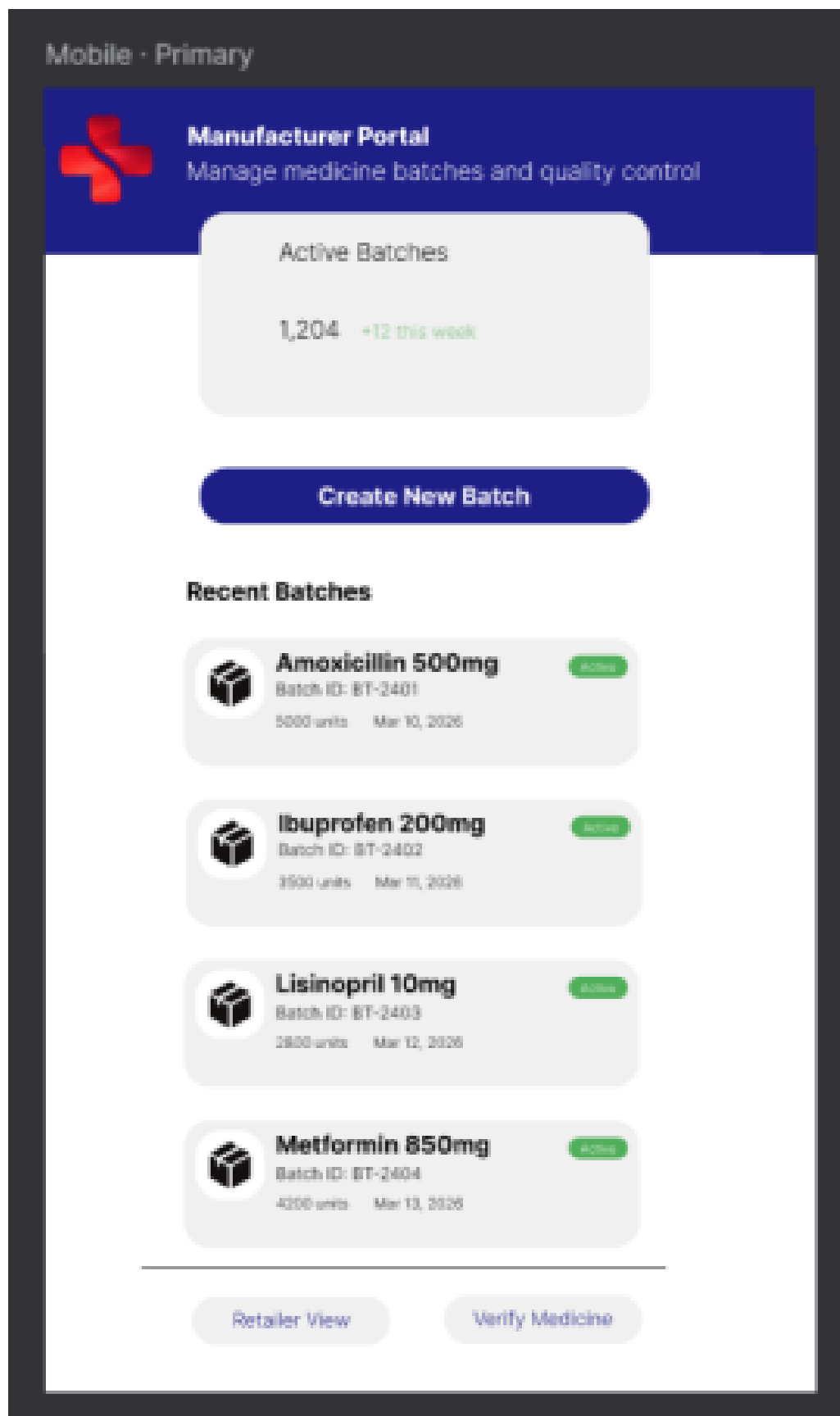


Figure 3.17: Figure 3.8.1: Manufacturer Portal — Wireframe

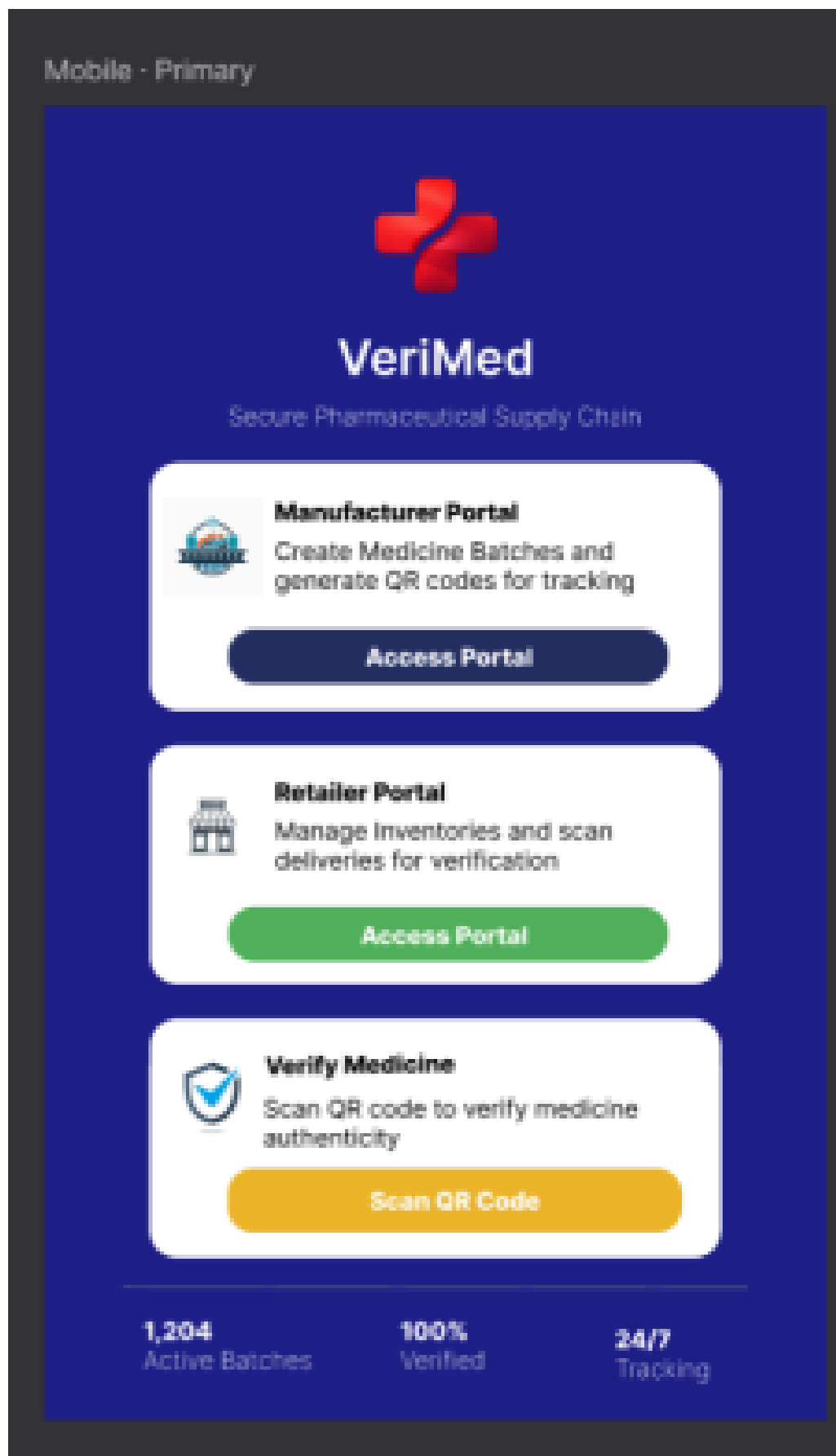


Figure 3.18: *Figure 3.8.2: VeriMed Home Page — Wireframe*
Fundamentals of Software Engineering · Final Project Report

Mobile - Primary



VeriMed



Enter Full Name



Email Address



Enter Password

Sign Up

Already have an account ?

[Sign In](#)

Figure 3.19: *Figure 3.8.3: VeriMed SignUp Page — Wireframe*

4 Implementation

4.1 Development Environment

This VeriMed system was developed in an organized design and development process which is designed to develop an efficient healthcare management application.

4.1.1 Tools and Softwares

- **VS Code:** The IDE primarily used for coding.
- **Figma:** Used for prototypes and UI design.
- **Postman:** Used to perform checks on RESTful APIs.
- **Git/GitHub:** Used to maintain versions of code.

4.1.2 Programming Languages and Components Development

- **HTML5 & CSS3:** Used to build the website structure.
- **React.js:** Used to develop a React-based front-end application with components and hooks like `useState` and `useEffect`.
- **Python (Flask):** Used to develop the backend of the application.
- **MySQL:** MySQL database is used to store data properly in third normal form.

4.2 System Modules Description

4.2.1 Manufacturer Module

The Manufacturer component aids in initiating the secure supply chain. This component comprises a secure login feature that provides access to a dashboard where the users can enter information about the medicines. After submission, the system creates a hash function, which is then encoded to a Base64 QR code image and saved in the `medicine_batches` table.

4.2.2 Retailer Module

The “Scan to Receive” process in the Retailer module makes use of jsQR to scan the barcode via the camera of the device and retrieve batch information. The collected information is later uploaded to the FEFO inventory dashboard, wherein items nearing expiration are displayed in different colors.

4.2.3 Patient / Verification Module

This user-friendly module does not require any sign-in process for users to check the validity of medicines within seconds. It contains a scanner window along with an input section where users can enter their batch IDs. In case the batch ID is missing or out of date, the module will notify the users accordingly.

4.3 Frontend Implementation

The front-end was developed through React.js as a Single Page Application. This design of the website is supposed to adapt to various types of devices. Custom CSS modules were used to give the website its unique look. The website has also been made easy to navigate by implementing `react-router-dom`.

4.4 Backend Implementation

The backend is implemented using Flask which is set with RESTful API. There are many significant functionalities, some of which are:

- `POST /Login` — for requesting authentication

- `POST /createBatch` — for unique batch generation
- `GET /publicVerifyBatch` — to verify the batch

4.5 Database Implementation

Since MySQL was used, we were able to make the database in 3NF form. The structure of our database includes many tables like *users*, *medicines*, etc., with proper foreign keys maintained between the tables. Other database rules are also followed.

4.6 Integration of System Components

The integration is carried out by making asynchronous API requests (via `fetch` or `axios`) from the React front-end to the Flask back-end. Upon form submission in HTML, the data is validated by the React front-end application, and then the data undergoes the process of being inserted into or updated in the MySQL database through the Flask script, with JSON feedback sent back to update the user interface accordingly.

4.7 Key Functionalities

Features of Manufacturer Dashboard

- **“Create Batch” Form:** This dashboard includes a “Create Batch” form, through which the manufacturer can enter data like Formula name, number of batches, expiration date, etc.
- **QR Code Generation:** Once this form is filled in, the back end generates a unique code which becomes a QR code on the dashboard itself.
- **List of Active Batches:** Other than this, there is a list of all the batches generated by the manufacturer displayed on the dashboard.

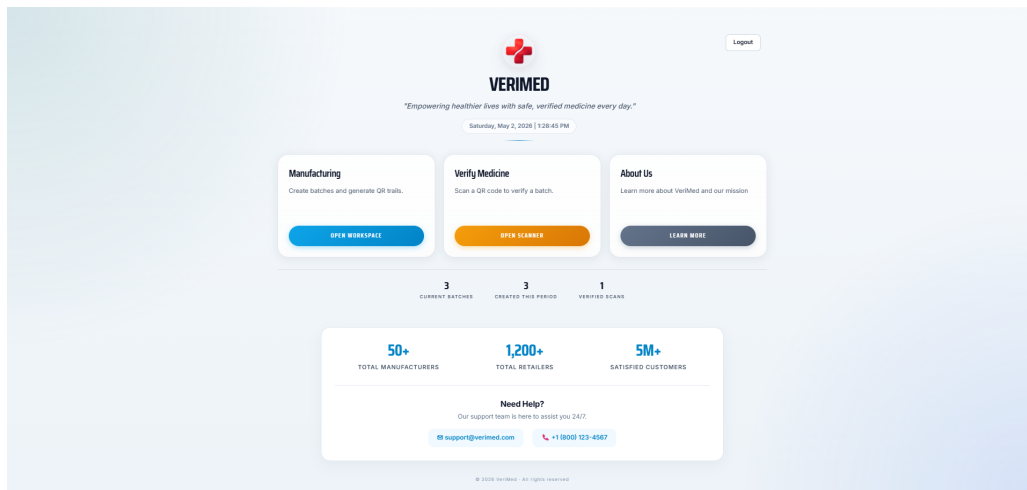


Figure 4.1: Figure 4.7.1: Manufacturer Homepage

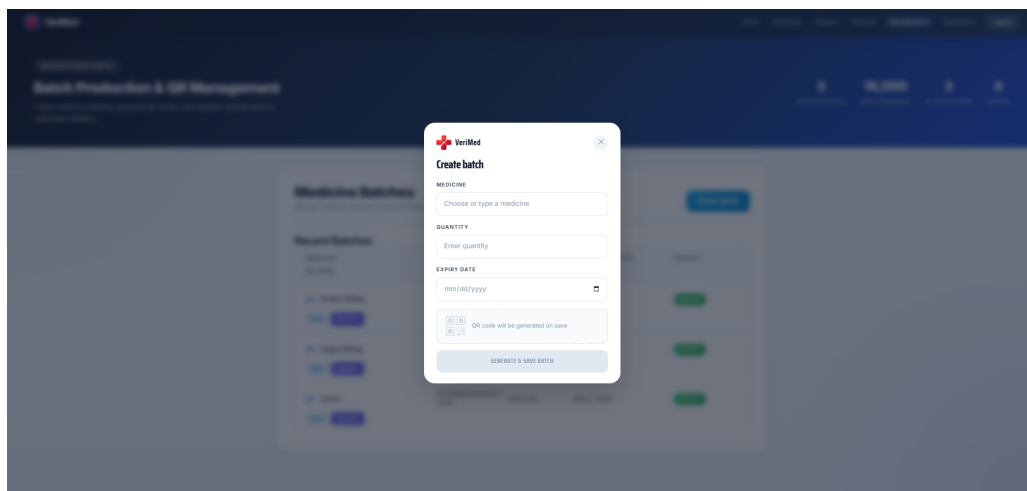


Figure 4.2: Figure 4.7.2: Batch Creation

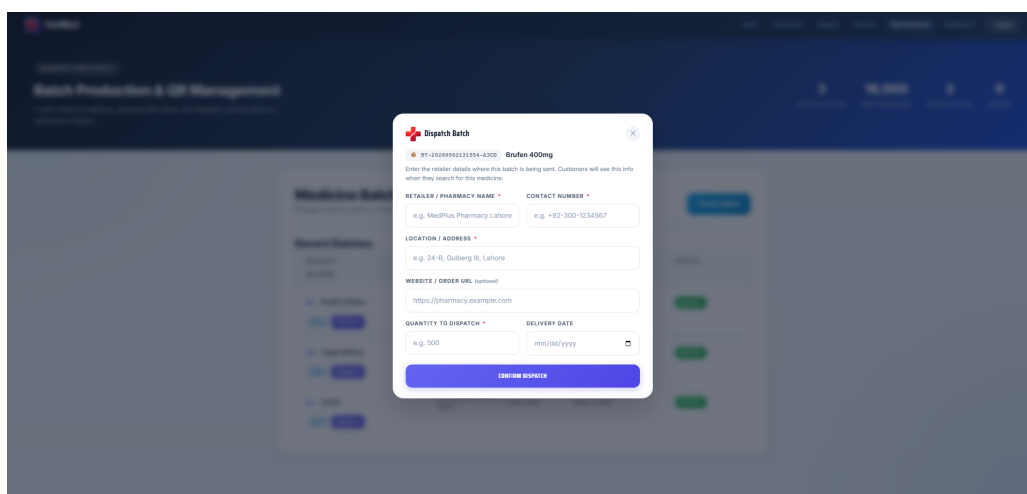


Figure 4.3: Figure 4.7.3: Batch Dispatch

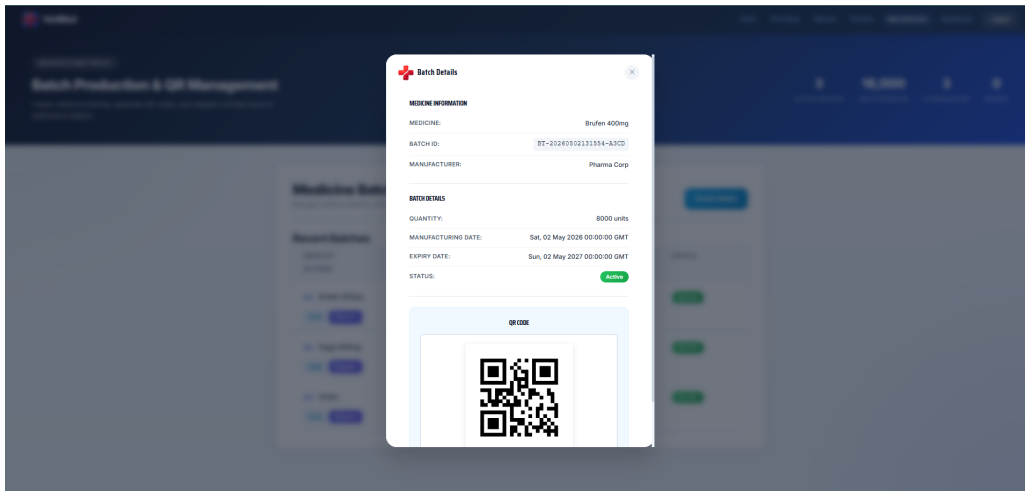


Figure 4.4: Figure 4.7.4: Batch Details

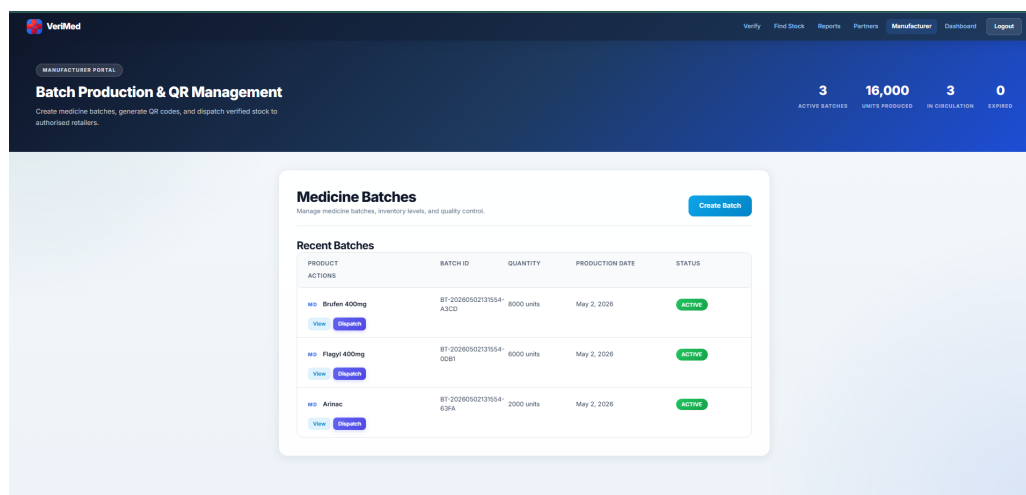


Figure 4.5: Figure 4.7.5: Manufacturer Portal

Retailer Dashboard & Its Features

- Scan to Initiate Procedure:** Leveraging the functionalities offered by the jsQR plugin, the retailer scans the QR code created by the manufacturer utilizing the device camera. Immediately afterward, a fast “handshake” session starts whereupon authentication of the product occurs, following which it gets added to the list of inventories.
- FEFO Based Inventory Control System:** The products are classified in accordance with the FEFO rules, and the medicines in the list show the following status:
 - Green:** A valid item with sufficient shelf life.
 - Yellow:** Expiry Warning within six months.
 - Red:** Expired stock that cannot be sold as per system requirements.

- **Stock Analytics:** Retailers get access to the total number of units sold compared to units in stock.

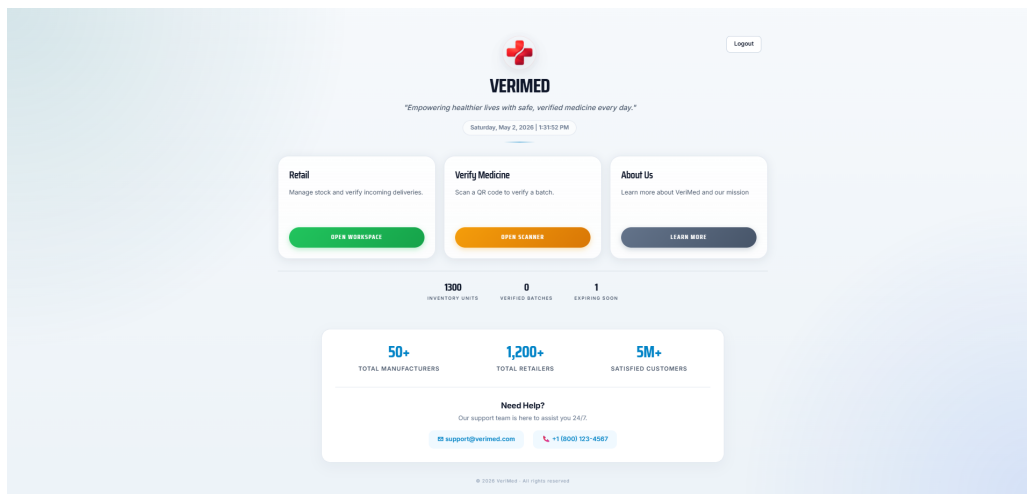


Figure 4.6: Figure 4.7.6: Retailer's Homepage

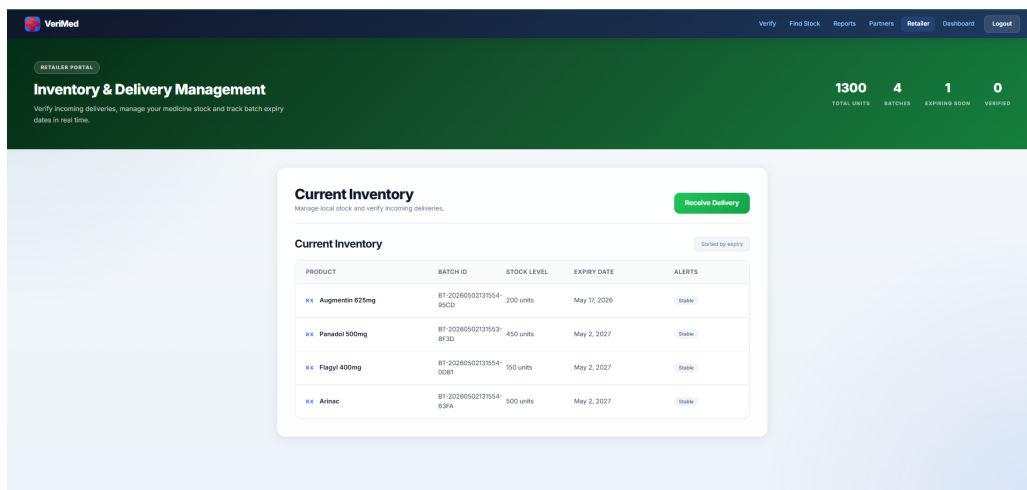


Figure 4.7: Figure 4.7.7: Retailer Portal

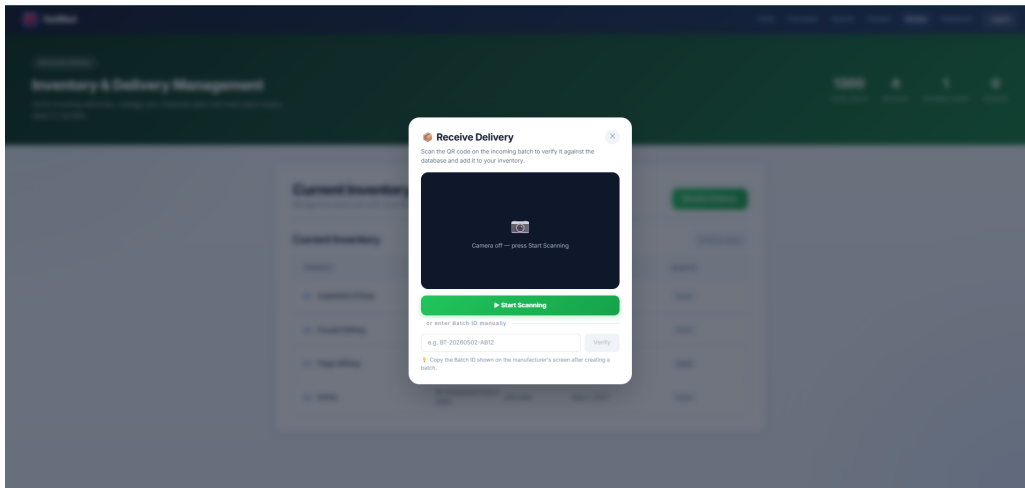


Figure 4.8: *Figure 4.7.8: Receive Delivery*

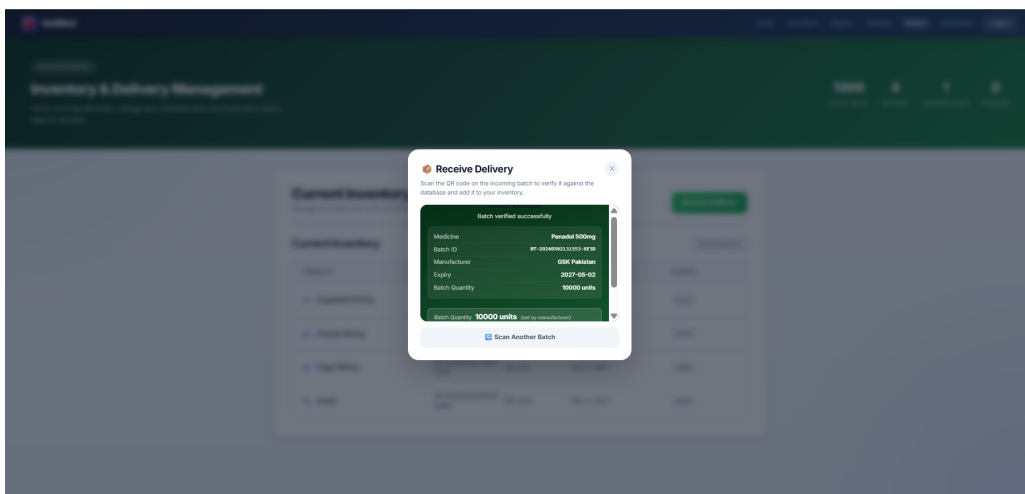


Figure 4.9: *Figure 4.7.9: Verification Successful*

Customer Dashboard and Functions

- **Verify Medicine Public Interface:** This screen presents customers with a user-friendly Verify Medicine screen that consists of an in-built camera scanner as well as a Batch ID entry box.
- **Provenance Details:** Successful scan results in displaying a “Genuine” badge along with full provenance details including the name of the manufacturer, license details, and expiration date of the medicine.
- **Warning Screen:** In case there is any anomaly in the code inputted in the system, the user interface automatically switches to a warning screen that contains a reporting form whereby the consumer can report details of the pharmacy location and store name which will be captured in the `qr_verifications` table.

- **Medicine Finder Screen:** Customers are able to find out which pharmacies have specific medicines in stock near them.

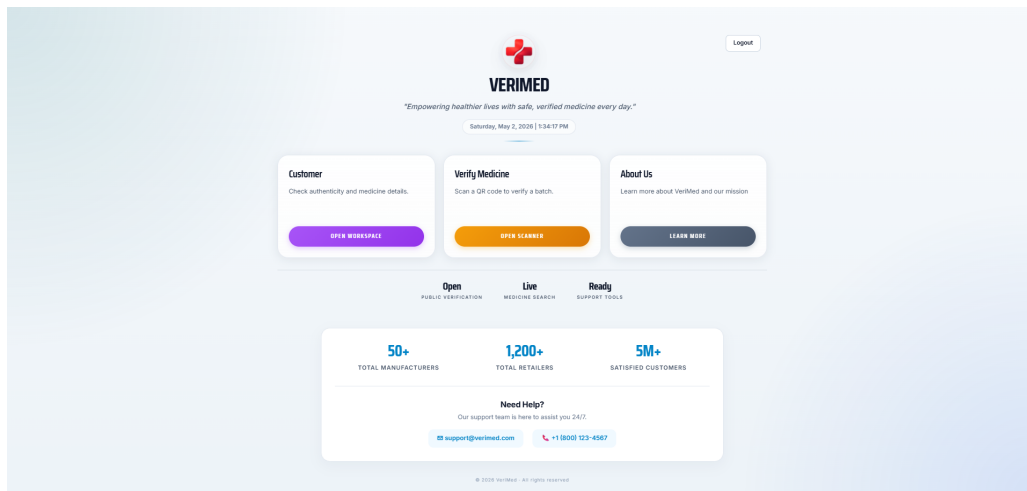


Figure 4.10: Figure 4.7.10: Customer's Homepage

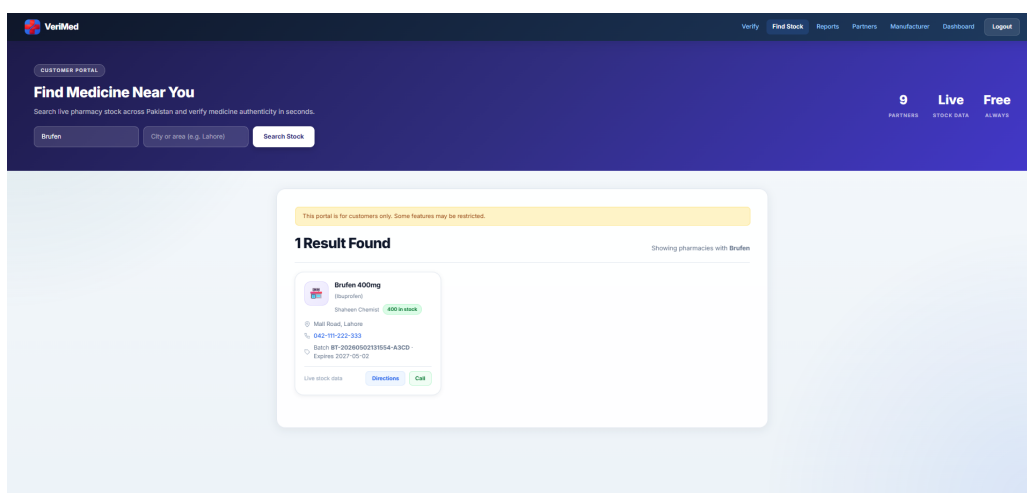


Figure 4.11: Figure 4.7.11: Search Medicine

Public Verification Screen: An on-the-go verification screen that instantly lets the consumer know whether the item is “Genuine” or a “Counterfeit.”

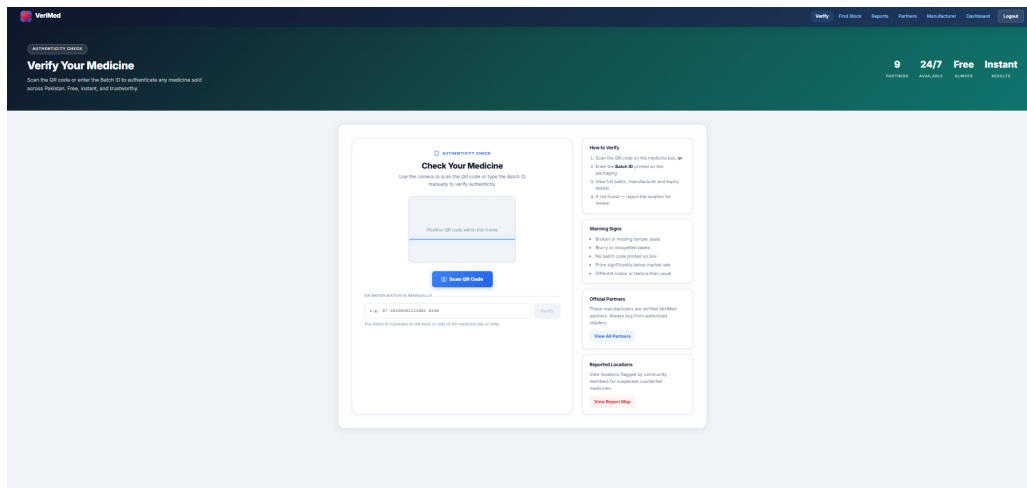


Figure 4.12: Figure 4.7.12: Public Verification Portal

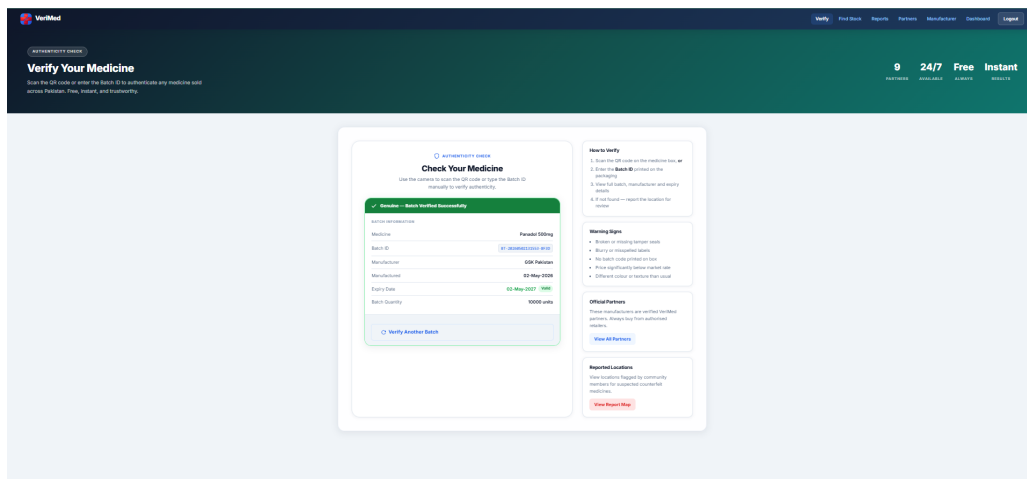


Figure 4.13: Figure 4.7.13: Successful Verification

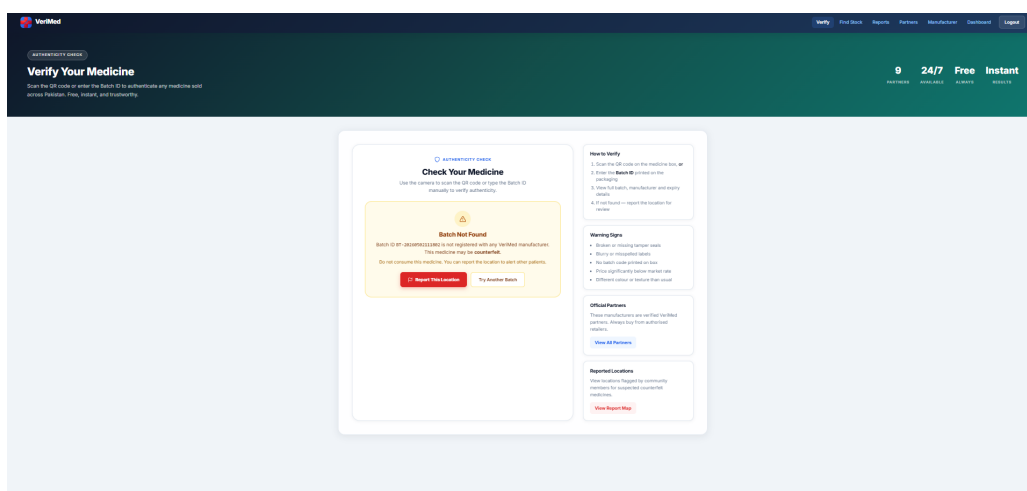


Figure 4.14: Figure 4.7.14: Unsuccessful Verification

Crowdsourced Reporting: A reporting feature in the module that allows people to flag suspicious areas where purchases are made.

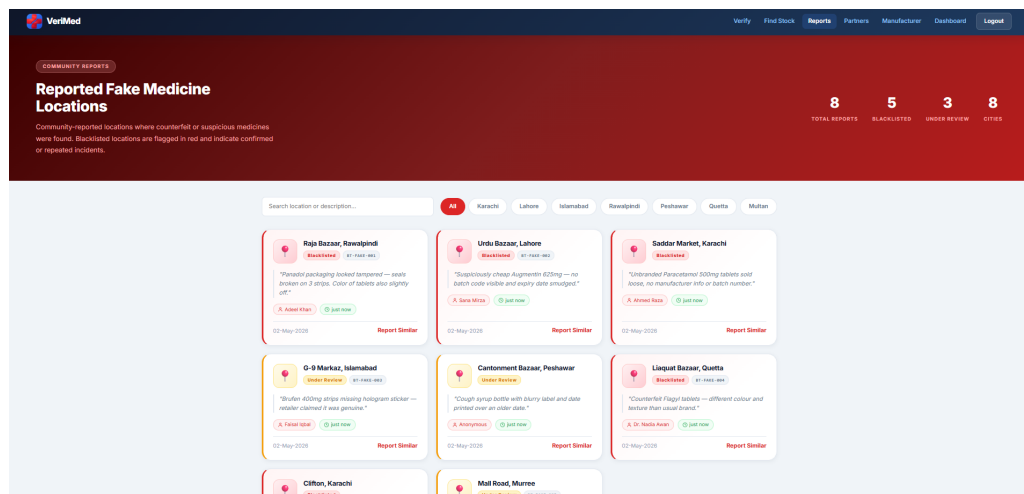


Figure 4.15: Figure 4.7.15: Reported Fake Medicine Locations

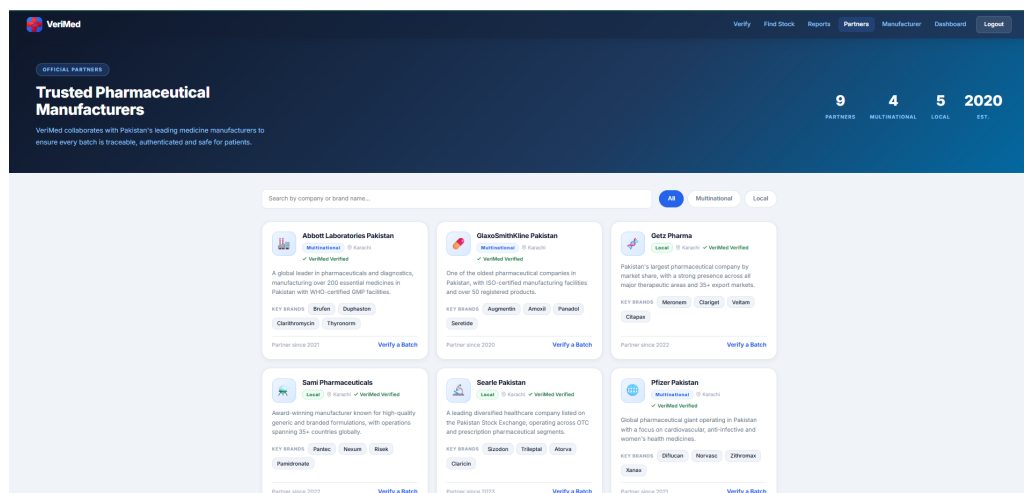


Figure 4.16: Figure 4.7.16: Trusted Pharmaceutical Partners

5 Testing & Evaluation

5.1 Testing Strategy

The testing strategy for VeriMed was planned such that the integrity of the medical supply chain is validated by ensuring that all the components of the app function well and provide accurate results.

5.1.1 Levels of Testing

- **Unit Test:** The tests focused on checking the functionality of the functions in the Flask back-end, such as the hashing of the message and the `extractBatchId` tool.
- **Integration Test:** This test ensured the proper functioning of the connection between the React front-end and MySQL database using Flask API services, whereby the form data input into forms (`Signup.jsx`, `Manufacturer.jsx`) is saved in the database.
- **System Testing:** The system test was done using the full flow of batch creation by manufacturers to the final authentication process of the batches by patients.

5.1.2 Methodology

- **Functional Testing:** The testing procedure that ensured whether functionalities such as QR scanner, role-based login, etc., met the requirements.
- **Boundary Value Analysis (BVA):** The method adopted to test the critical parameters like Batch Size and Expiry Date.

5.2 Test Cases and Results

The following test cases were executed to validate the system’s core functionalities.

Test ID	Test Description	Expected Result	Result
TC-01	User Login with valid credentials	Successful redirect to role-specific dashboard	Pass
TC-02	Batch Creation with valid data	QR code generated and saved to DB	Pass
TC-03	QR Code Scan (Authentic Batch)	System displays “Genuine” status and details	Pass
TC-04	QR Code Scan (Fake/Invalid Hash)	System displays “Counterfeit” warning	Pass
TC-05	Inventory FEFO Sorting	Stock sorted by earliest expiry date	Pass
TC-06	Search Medicine by City	Returns list of local verified retailers	Pass
TC-07	Submit Fake Report	Report details logged in <code>qr_verifications</code>	Pass
TC-08	Unauthorized Access to /mfg	Redirect to login / 403 Forbidden	Pass
TC-09	Password Reset Request	Confirmation message “Reset Link Sent” displayed	Pass
TC-10	Empty Form Submission	Validation error message displayed	Pass

Table 5.1: *Test Cases and Results*

Boundary Value Analysis (BVA) for Critical Inputs

Field	Input Type	Value Tested	Expected Result	Status
Quantity	Minimum	1	Accepted	Pass
Quantity	Zero/Negative	0, -5	Error: “Invalid Quantity”	Pass
Expiry Date	Past Date	2023-01-01	Error: “Date cannot be in past”	Pass
Expiry Date	Far Future	2050-12-31	Accepted	Pass

Table: Boundary Value Analysis (BVA) for Critical Inputs

5.3 System Validation

According to the results of testing, the system is working properly in respect of the following functionalities:

- **FR-01 (Authentication):** The users are able to authenticate and get access to their roles using role-based access along with a secure session management technique (JWT).
- **FR-02 (Traceability):** Hash values are generated for each batch which are traceable and successfully validated.
- **FR-03 (Inventory Management):** Validation through the automated FEFO badge system and inventory stock updates.
- **FR-04 (Public Safety):** Validation through the counterfeit warning and location reporting system.

5.4 Performance Evaluation

For the performance tests, the following functions of the system were assessed for their response time:

- **Database Queries:** For searching the pharmacy locally and for inventory searches, all were found to be less than 0.5 seconds thanks to optimized MySQL indexing.
- **QR Code Creation:** Conversion of batch details into a Base64 image in less than one second.

5.5 User Feedback

Based on the initial feedback received internally, the “Glassmorphism” user interface design enhanced the usability for the retailers who had large inventories. External feedback from users will be sought in the upcoming development stages to enhance the community reporting feature.

6 Conclusion & Future Work

6.1 Summary of the Project

The problem of fake drugs, which is one of the most important problems on a global scale, is solved through the VeriMed initiative using an information system based on cryptography of QR codes together with a central relational ledger. The solution lies in connecting the missing link between producers, retail stores, and end users through a reliable database of unique digital IDs of every drug unit, allowing for instant identification through a smartphone camera.

6.2 Achievements

The project achieved many of the major components and capabilities outlined in this report:

- **Successful Portal Deployment:** There are three completely separate and entirely discrete role-based portals (Manufacturer, Retailer, and Customer), each with business logic and security specific to that component.
- **Secure Cryptographic Ledger:** A very strong backend was implemented that generates batch hashes and completely automatically saves them to a 3NF MySQL database so nothing can be lost.
- **Real-time Verification:** Implemented jsQR for client-side scanning, enabling an instantaneous authenticity check without requiring proprietary hardware.
- **Automated FEFO Inventory:** An automated “First-Expired-First-Out” management system has been implemented and is completely integrated into the pharmacy,

which alerts pharmacists to upcoming expirations, directly impacting patient safety.

6.3 Limitations

While the current iteration of VeriMed meets all core functional requirements, certain limitations exist:

- **Localhost Deployment:** Current deployment is limited to a local network environment, thereby making it not immediately usable in any global supply chain.
- **Centralized Database:** The entire architecture is completely built on a digitally centralized MySQL database, which can lead to a single point of failure and a serious threat of administrative-level data tampering.
- **Manual Reporting:** The reporting format works, but instead of being automated by geolocation it depends on customers manually entering location data.

6.4 Future Enhancements

The following future features are aimed at making VeriMed a more promising, resilient, and scalable platform:

- **Cloud Deployment:** The deployment of the Flask backend and the MySQL database in cloud service providers like AWS or Heroku to provide extensive global accessibility and high availability.
- **Decentralized Blockchain Ledger:** Replace with a Blockchain-based, decentralized distributed ledger for batch history to eliminate single points of failure and provide an immutable, tamper-proof history of every medicine batch.
- **Machine Learning Integration:** Predictive analytics to identify possible counterfeit “hotspots” based on crowdsourced reporting data.
- **Mobile App Expansion:** Create native Android and iOS apps to deliver a seamless, optimized scanning experience to consumers in rural markets.

A Source Code Snippets

Below are all the snippets that represent the core logical modules of VeriMed system, extracted from the already implemented Flask backend and React frontend.

A.1 Backend: Cryptographic Batch & QR Generation (app.py)

The following snippet demonstrates the cryptographic batch creation logic including QR code generation for physical labeling.

```
1 @app.route('/createBatch', methods=['POST'])
2 @jwt_required()
3 def create_batch():
4     data = request.get_json()
5     batch_id = data.get('batch_id')
6     medicine_id = data.get('medicine_id')
7     manufacturer_id = get_jwt_identity() # Secured via JWT
8     expiry_date = data.get('expiry_date')
9
10    # Generate cryptographic hash for the QR code
11    qr_data = f"Batch:{batch_id}|Med:{medicine_id}|Exp:{
expiry_date}"
12    qr_hash = hashlib.sha256(qr_data.encode()).hexdigest()
13
14    # Generate QR Code Image as Base64
15    qr = qrcode.make(qr_hash)
16    buffered = BytesIO()
17    qr.save(buffered, format="PNG")
```

```
18     qr_base64 = base64.b64encode(buffered.getvalue()).decode()
19
20     # Save to medicine_batches table
21     new_batch = MedicineBatch(
22         batch_id=batch_id,
23         medicine_id=medicine_id,
24         manufacturer_id=manufacturer_id,
25         qr_code_base64=qr_base64,
26         batch_status='active'
27     )
28     db.session.add(new_batch)
29     db.session.commit()
30     return jsonify({"message": "Batch Created", "qr": qr_base64
31                     }), 201
```

Listing A.1: Backend: Cryptographic Batch & QR Generation (*app.py*)

A.2 Database: FEFO Inventory Sorting Query (*database.sql*)

```
1  -- Query to fetch inventory sorted by expiration date (FEFO)
2  SELECT
3      m.medicine_name,
4      b.batch_id,
5      ri.quantity_in_stock,
6      b.expiry_date,
7      DATEDIFF(b.expiry_date, CURDATE()) AS days_to_expiry
8  FROM retailer_inventory ri
9  JOIN medicine_batches b ON ri.batch_id = b.id
10 JOIN medicines m ON b.medicine_id = m.id
11 WHERE ri.retailer_id = %s AND ri.quantity_in_stock > 0
12 ORDER BY b.expiry_date ASC;
```

Listing A.2: FEFO Inventory Sorting Query (*database.sql*)

A.3 Frontend: Real-Time QR Scanner Integration (*Retailer.jsx*)

```
1  const scanQRCode = () => {
2    if (videoRef.current.readyState === videoRef.current.
      HAVE_ENOUGH_DATA) {
3      const canvas = canvasRef.current;
4      const context = canvas.getContext("2d");
5      context.drawImage(videoRef.current, 0, 0, canvas.width,
        canvas.height);
6
7      const imageData = context.getImageData(0, 0, canvas.width,
        canvas.height);
8      const code = jsQR(imageData.data, imageData.width,
        imageData.height);
9
10     if (code) {
11       console.log("Found QR code:", code.data);
12       handleVerification(code.data); // Sends hash to Flask API
13     }
14   }
15   requestAnimationFrame(scanQRCode);
16 };
```

Listing A.3: Real-Time QR Scanner Integration (*Retailer.jsx*)

B User Manual

B.1 For Manufacturers: Generating a Batch

1. **Login:** You need to login through the Manufacturers portal via your credentials.
2. **Data Input:** Go to the Create Batch page and enter the formula, batch size, and expiration date.
3. **Create QR:** Click on the “Generate QR” button and download the image file. Print the QR code on the package of your batch.

B.2 For Retailers: Accepting Stock

1. **Scanning QR:** Select the Retailer Dashboard and click the Accept Delivery button.
2. **Verification:** Using the QR Code, the scanner will emit a green light if the batch is registered.
3. **Inventory Update:** Click on “Add to Inventory” to update the FEFO stock inventory automatically.

B.3 For Patients: Verifying Medicine

1. **Scanner:** Visit the publicly accessible VeriMed website on your phone’s web browser without logging in.
2. **Scan Medicine:** Scan the barcode of the medicine.

3. **Verification:** The verification of the medicine will be done by analyzing its manufacture and expiry date.

C Screens / UI Designs

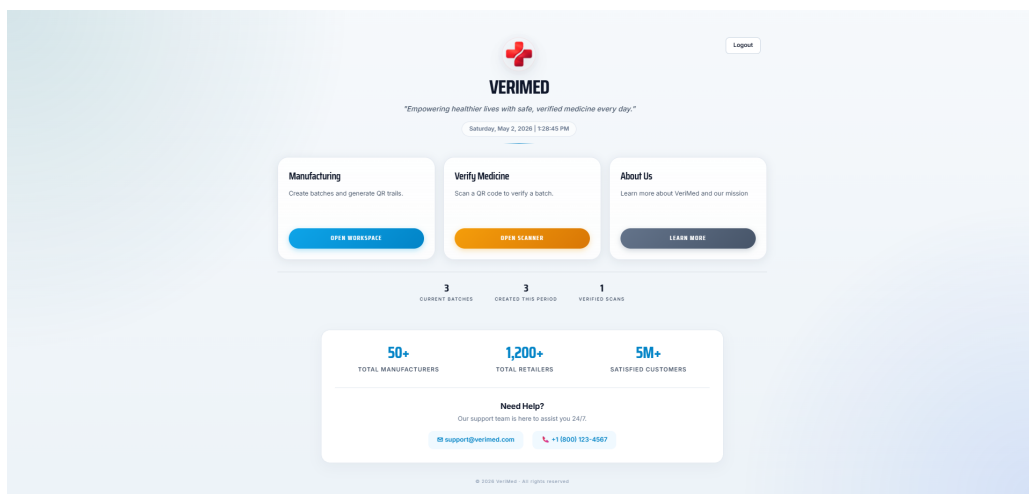


Figure C.1: *Figure C-1: VeriMed Home Page for Manufacturer*

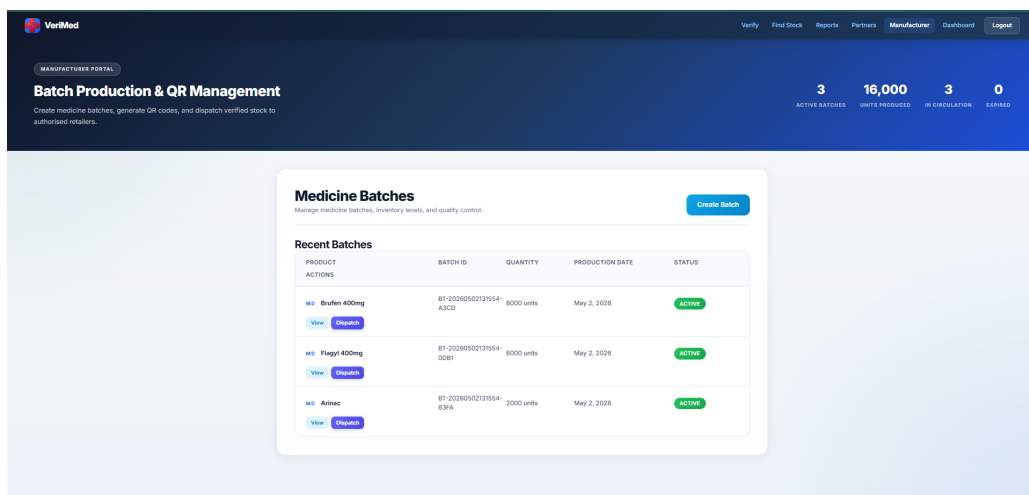


Figure C.2: *Figure C-2: Manufacturer Portal*

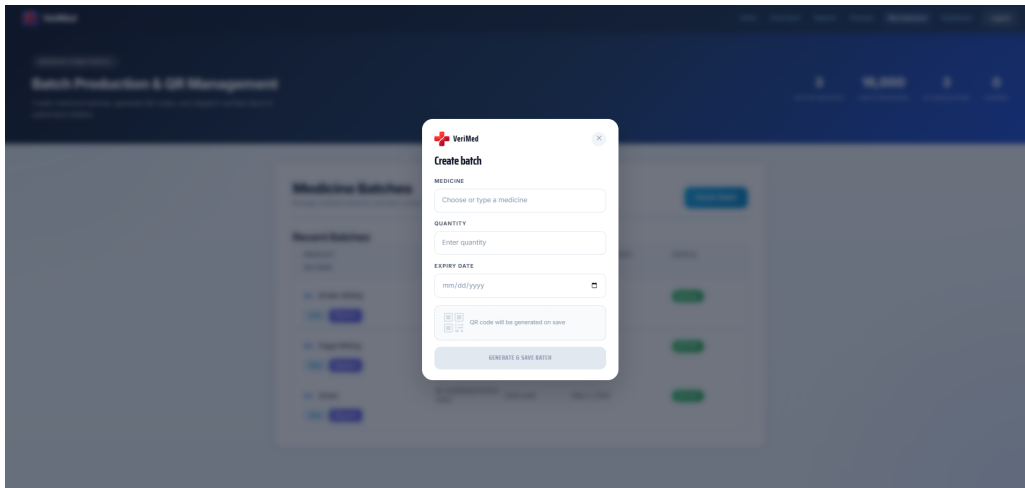


Figure C.3: *Figure C-3: Medicine Batch Creation for Manufacturer*

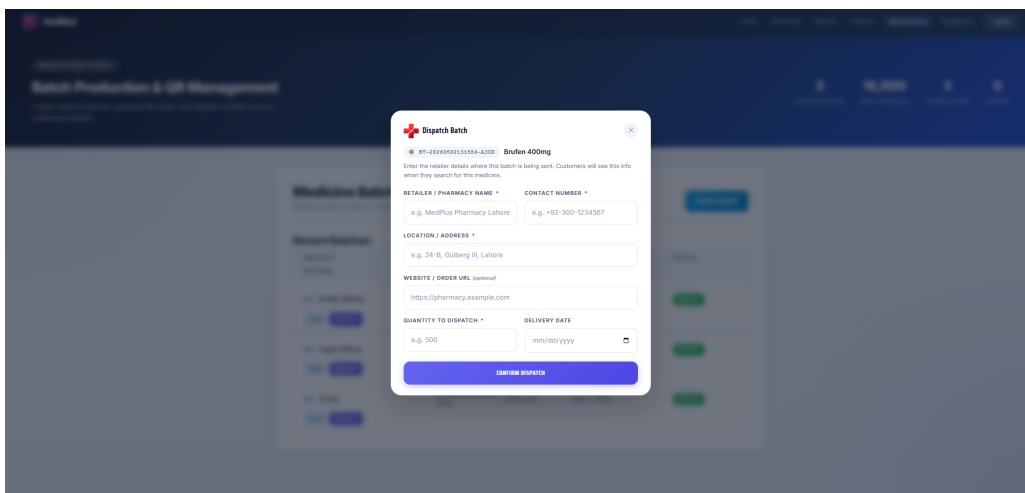


Figure C.4: *Figure C-4: Batch Dispatch for Manufacturer*

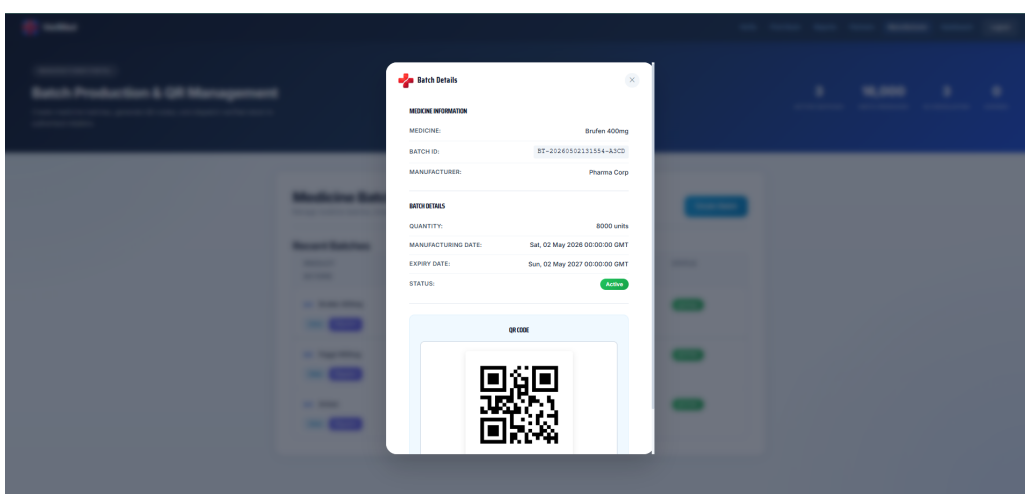


Figure C.5: *Figure C-5: Batch Details Display for Manufacturer*

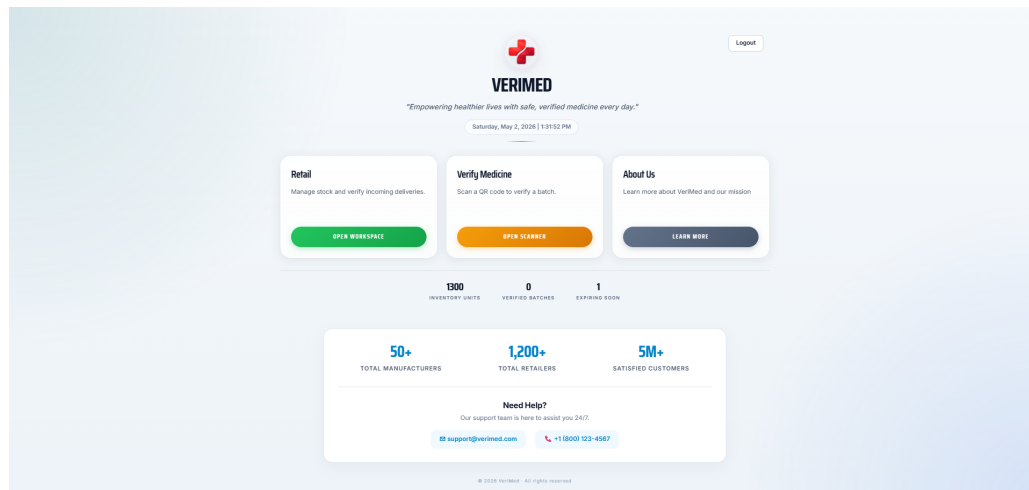


Figure C.6: Figure C-6: VeriMed Homepage for Retailer

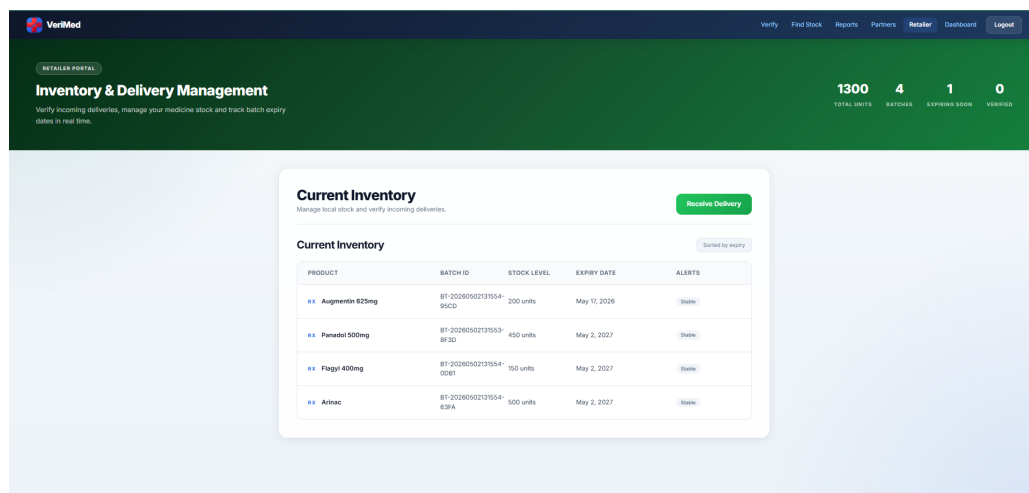


Figure C.7: Figure C-7: Retailer Portal

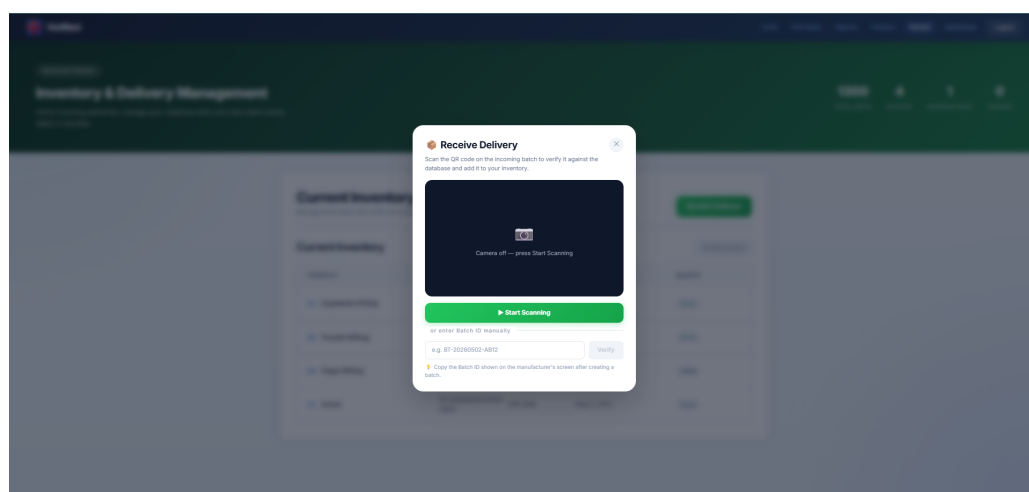


Figure C.8: Figure C-8: Retailer Delivery Verification

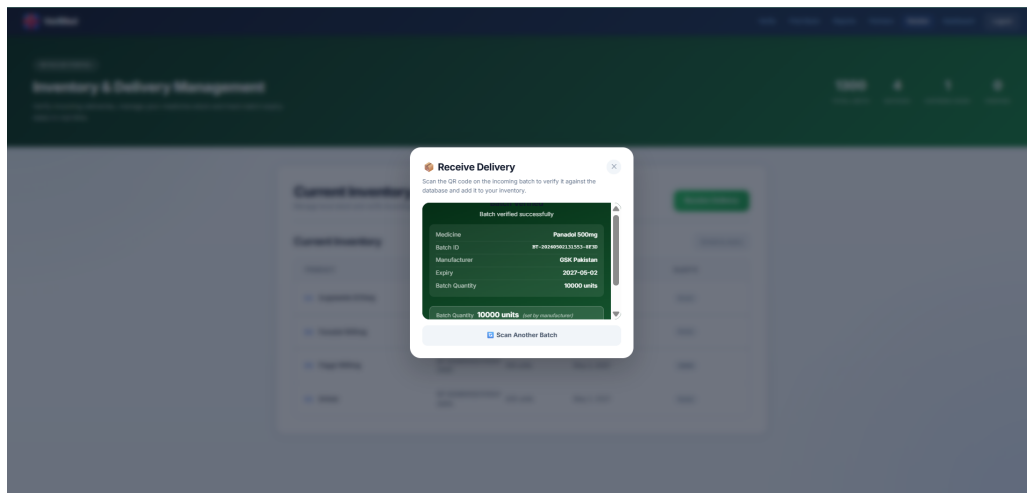


Figure C.9: Figure C-9: Retailer Delivery Verification

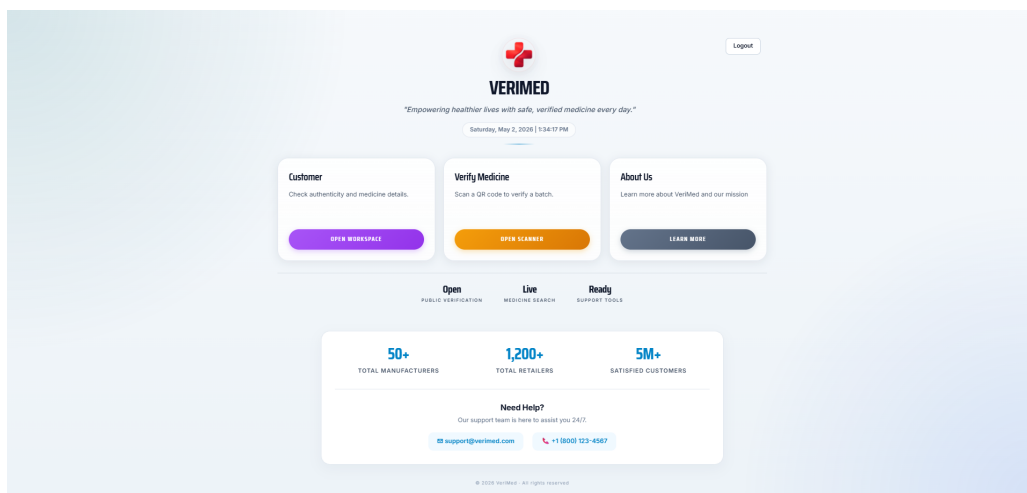


Figure C.10: Figure C-10: VeriMed Homepage for Customers

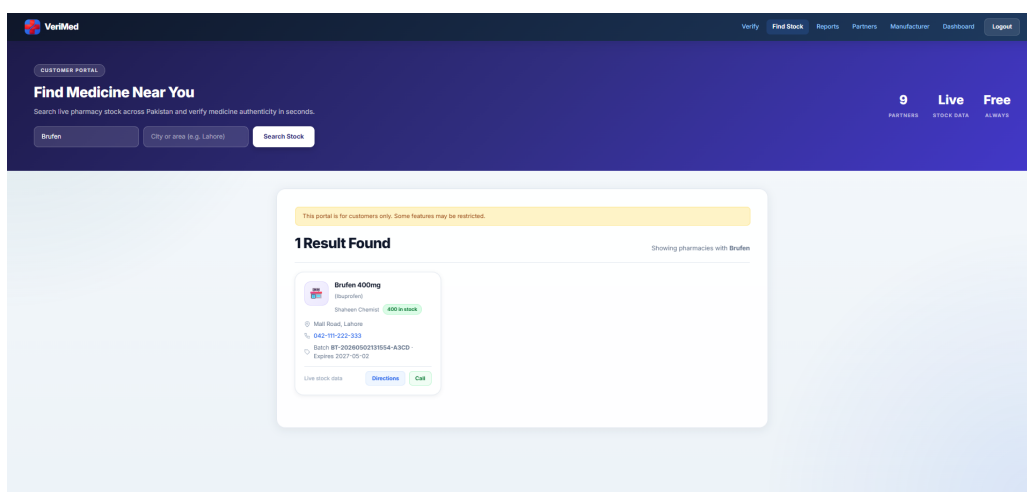


Figure C.11: Figure C-11: Search for Medicine

The screenshot shows the 'Verify Your Medicine' page on the VeriMed website. The page has a dark green header with navigation links: 'Verify', 'Find Batch', 'Reports', 'Partners', 'Manufacturer', 'Dashboard', and 'Logout'. On the right side of the header, there are statistics: '9 PARTNERS', '24/7 AVAILABLE', 'Free', and 'Instant RESULTS'. The main content area is titled 'Check Your Medicine' with a sub-header 'Verify Your Medicine'. It instructs users to 'Scan the QR code or enter the Batch ID to authenticate any medicine and access Patient, Price, history, and trustworthiness.' The central form has a 'Check Your Medicine' section with a 'Scan QR Code' button and a 'Batch ID' input field. To the right, there are three informational boxes: 'How to Verify' (listing steps 1-4), 'Warning Signs' (listing signs like broken packaging, blurry text, etc.), and 'Official Partners' (listing trusted manufacturers). At the bottom right, there is a 'Report Suspicious Counterfeit' link.

Figure C.12: Figure C-12: Medicine's Authenticity Verification

The screenshot shows the 'Verify Your Medicine' page with a successful verification result. The central form now displays a green 'Success - Batch Verified Successfully' message. Below this, it lists the verified details: 'Batch ID: 12345678901234567890', 'Manufacturer: CDE Pharma', 'Expiry Date: 02-May-2027', and 'Batch Quantity: 10000 units'. A 'Verify Another Batch' button is visible at the bottom of the form. The right side of the page remains the same, with 'How to Verify', 'Warning Signs', 'Official Partners', and 'Report Suspicious Counterfeit' sections.

Figure C.13: Figure C-13: Successful Verification

The screenshot shows the 'Verify Your Medicine' page with a 'Report Suspicious Counterfeit' form open. The form is titled 'Report Suspicious Counterfeit' and includes fields for 'Location of purchase' (e.g., 'New Delhi, India'), 'Description' (e.g., 'The medicine is expired'), and 'Your Name (optional)'. There are 'Submit Report' and 'Cancel' buttons. The right side of the page remains the same, with 'How to Verify', 'Warning Signs', 'Official Partners', and 'Report Suspicious Counterfeit' sections.

Figure C.14: Figure C-14: Counterfeit Report

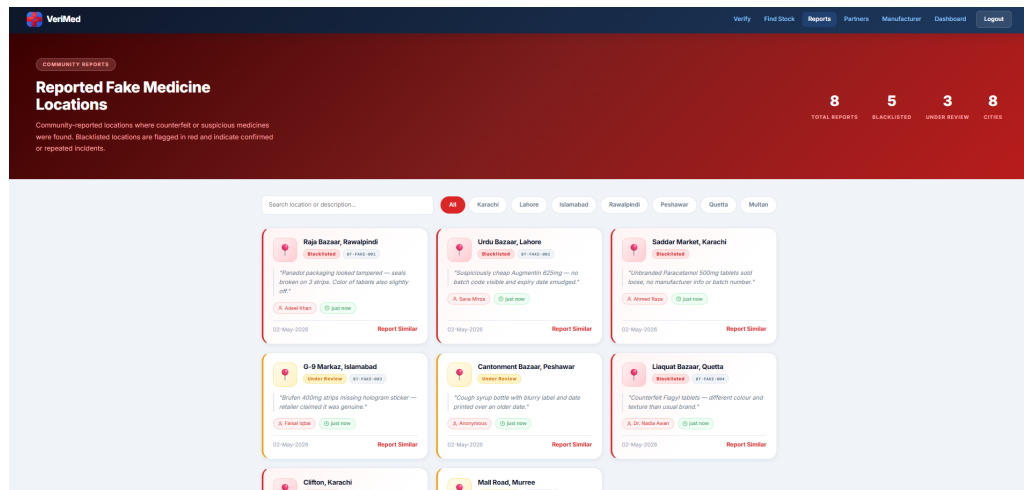


Figure C.15: Figure C-15: Reported Fake Medicine Locations

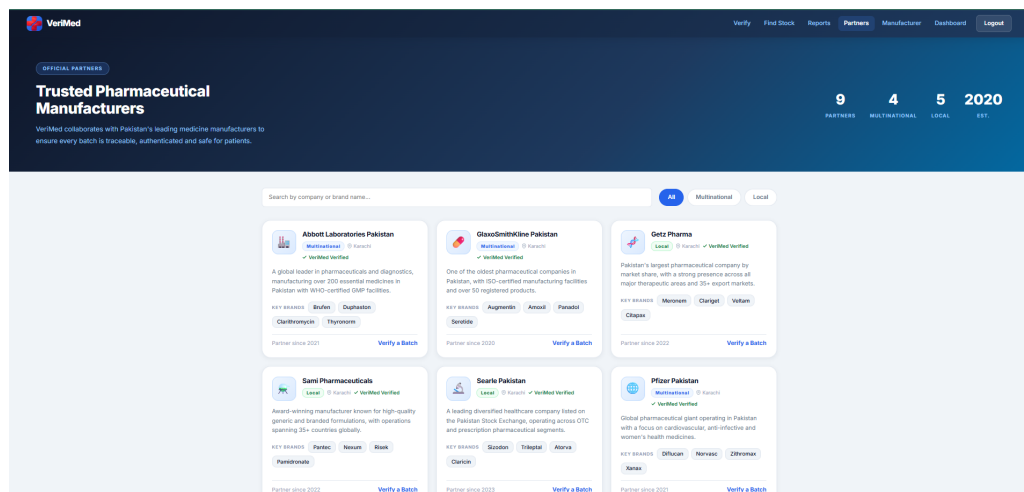


Figure C.16: Figure C-16: Trusted Partners

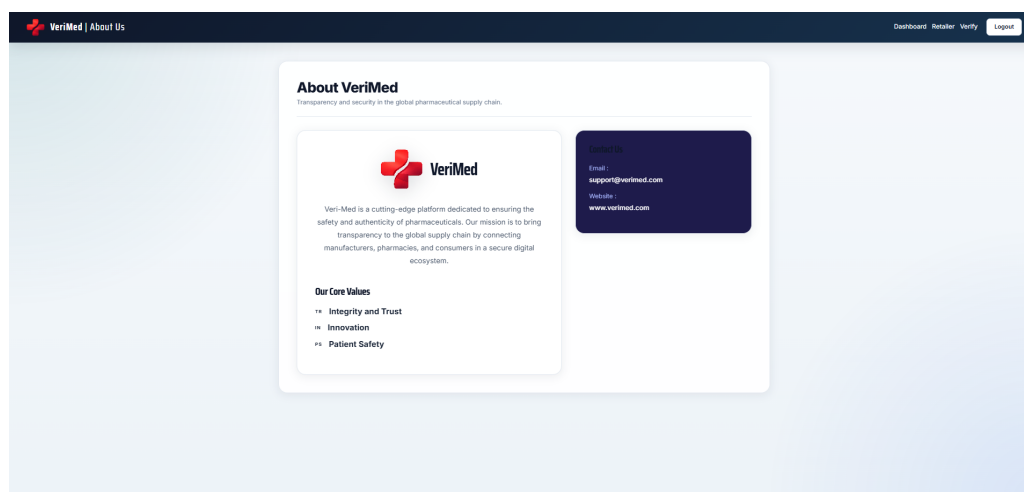


Figure C.17: Figure C-17: About VeriMed

D Additional Diagrams

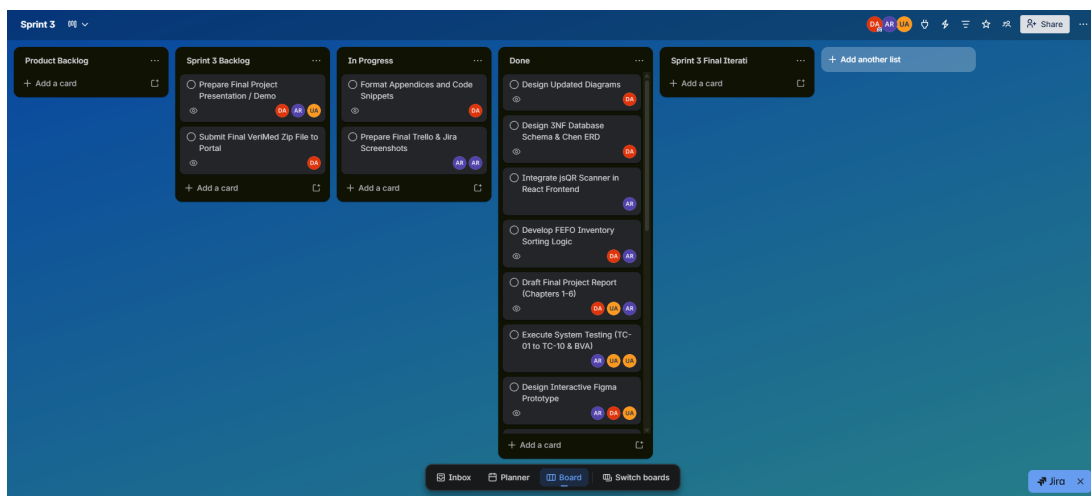


Figure D.1: *Figure D-1: Trello — Project Backlog and Progress Track*